

GESTIONARE GRADINA BOTANICA

Moldovan Dana
grupa 30231

Enunț problema:

Dezvoltați o aplicație software care poate fi utilizată într-o grădină botanică. Aplicația va avea 4 tipuri de utilizatori: vizitator, angajat, manager și administrator.

Utilizatorii de tip vizitator pot efectua următoarele operații fără autentificare:

- ❖ Vizualizarea listei tuturor plantelor din grădina botanică sortată după tip și specie (vizualizarea include și redarea unor imagini cu toate plantele din grădina botanică; între 1 și 3 imagini pentru fiecare plantă);
- ❖ Filtrarea listei plantelor după următoarele criterii: tip, specie, plante carnivore, zona grădină botanică;
- ❖ Căutarea unei plante după denumire sau specie.

Utilizatorii de tip angajat al grădinii botanice pot efectua următoarele operații după autentificare:

- ❖ Toate operațiile permise utilizatorilor de tip vizitator;
- ❖ Operații CRUD în ceea ce privește persistența plantelor, dar și a exemplarelor din fiecare specie din grădina botanică;
- ❖ Salvare liste cu plantele în mai multe formate: csv, json, xml, doc.

Utilizatorii de tip manager al grădinii botanice pot efectua următoarele operații după autentificare:

- ❖ Toate operațiile permise utilizatorilor de tip angajat;
- ❖ Vizualizarea unor statistici (minim 3) legate de plantele din grădina botanică utilizând grafice și salvarea acestora în fișier Word.

Utilizatorii de tip administrator pot efectua următoarele operații după autentificare:

- ❖ Toate operațiile permise utilizatorilor de tip vizitator;
- ❖ Operații CRUD pentru informațiile legate de utilizatorii care necesită autentificare;
- ❖ Vizualizarea listei utilizatorilor care necesită autentificare, salvarea acestora în format csv și filtrarea acestora după tipul utilizatorilor;
- ❖ Notificarea fiecărui utilizator care necesită autentificare prin cel puțin 2 variante (email, SMS, WhatsApp, etc.) la orice modificare a informațiilor de autentificare aferente acelui utilizator.

Interfața grafică a aplicației client va fi disponibilă în cel puțin 3 limbi de circulație internațională.

INSTRUMENTE UTILIZATE

Aplicația utilizează un stack tehnologic modern, structurat pe mai multe niveluri:

Backend — Java 21 cu Spring Boot 3.2, Spring Security + JWT pentru autentificare, Spring Data JPA (Hibernate) pentru persistență, și MySQL 8 ca bază de date.

Comunicarea între microservicii se realizează prin REST API cu RestTemplate, rutate printr-un API Gateway implementat cu Spring Cloud Gateway 4.0

Frontend — React 19 cu suport multilingv prin i18next (română, engleză, maghiară), vizualizare date prin Chart.js, și comunicare HTTP prin Axios.

Funcționalități auxiliare — Apache POI 5.2.3 pentru generarea documentelor Word, JFreeChart pentru grafice server-side, Spring Boot Mail pentru notificări email, Twilio API pentru notificări WhatsApp, și Maven pentru gestionarea dependențelor.

Mediu de dezvoltare — IntelliJ IDEA 2024

JUSTIFICAREA LIMBAJULUI DE PROGRAMARE ALES

Backend — Java 21

Java a fost ales ca limbaj principal de backend datorita integrării native cu Spring Boot, care simplifică semnificativ dezvoltarea microserviciilor. Spring Security oferă suport robust pentru autentificare JWT și control al accesului bazat pe roluri, iar portabilitatea JVM garantează că aplicația rulează consistent pe orice platformă. Java 21 aduce îmbunătățiri notabile de performanță și gestionare a memoriei, iar bibliotecile disponibile (Apache POI, JFreeChart, JWT) acoperă toate nevoile proiectului.

Frontend — React 19

React a fost ales pentru arhitectura sa bazată pe componente reutilizabile, care facilitează întreținerea și extinderea interfeței. Virtual DOM-ul asigură o randare rapidă și eficientă și a permis implementarea rapidă a funcționalităților necesare. Interfața este responsivă și adaptabilă diferitelor dispozitive.

Alternative precum Angular sau Vue ar fi putut fi utilizate, dar React oferă cel mai bun echilibru între flexibilitate, performanță și dimensiune. În plus, componentele funcționale simplifică semnificativ gestionarea stării și a efectelor laterale.

JUSTIFICAREA TIPULUI DE COMUNICARE ÎNTRE MICROSERVICII

Comunicarea între microservicii este realizată prin REST API, alegere justificată prin simplitatea protocolului HTTP, natura stateless a cererilor (fiecare cerere include token-ul JWT), și scalabilitatea independentă a fiecărui serviciu.

Toate cererile din frontend trec prin API Gateway, care centralizează rutarea, gestionarea CORS și validarea token-urilor JWT înainte de a direcționa cererea către microserviciul corespunzător. Astfel, frontend-ul nu comunică nicio

Exemplu de flux:

1. Frontend-ul trimite cerere → API Gateway (verifică JWT)
2. Gateway-ul routează → microserviciul corespunzător
3. Dacă este necesar (ex: export), microserviciul apelează → alt microserviciu via REST
4. Răspunsul se întoarce pe același drum către client

Avantaje:

- Stateless → fiecare cerere conține token-ul, scalabilitate orizontală
- Tehnologie agnostică → orice client poate consuma API-ul
- Caching → suport nativ la nivel HTTP dată direct cu serviciile interne.

Comunicarea între microservicii este realizată sincron prin REST API, folosind RestTemplate oferit de Spring Framework. Fiecare microserviciu care necesită date din alt serviciu face un apel HTTP către endpoint-ul corespunzător.

Dependențele concrete sunt următoarele: notification-export-service apelează plant-service pe GET /api/plants pentru a prelua lista plantelor la export, și pe GET /api/statistics/all pentru raportul Word; apelează de asemenea user-service pe GET /api/users/{id} pentru a obține datele utilizatorului (email, telefon) necesare notificărilor. La rândul său, user-service apelează plant-service pentru statistici, exemplar-service pentru filtrarea după zonă, și notification-export-service pentru a trimite cereri de export și notificări.

Token-ul JWT generat la autentificare este inclus în header-ul Authorization: Bearer la fiecare cerere HTTP trimisă de frontend. Când un microserviciu apelează intern alt microserviciu, token-ul original este extras din cererea inițială și propagat mai departe. Concret: frontend-ul trimite cererea către API Gateway cu token-ul, gateway-ul îl validează și îl transmite microserviciului, iar microserviciul îl extrage și îl folosește în apelurile interne. Extragerea se face prin RequestContextHolder în NotificationService.

Validarea JWT se realizează o singură dată, la nivel de API Gateway, înainte de rutare. Rutele publice (login, register, validate, GET /api/plants, GET /api/statistics/*, vizualizare imagini) nu necesită token. Toate rutele POST, PUT, DELETE, precum și rutele de export și notificări, necesită JWT valid. Dacă token-ul lipsește, este expirat sau invalid, cererea este respinsă cu 401 Unauthorized.

Fiecare microserviciu are propria configurație de securitate, însă validarea efectivă este delegată gateway-ului pentru a evita dubla verificare și a centraliza logica de securitate. Excepție face endpoint-ul /api/users/{id}, apelat intern de notification-export-service, care necesită token valid — iar notification-export-service este configurat să îl transmită.

BAZA DE DATE

Datele aplicației sunt distribuite în patru baze de date MySQL 8 complet independente, fiecare corespunzând unui microserviciu distinct.

Plant Service (8081) folosește baza de date b_plante, care conține tabela plante cu informații despre denumire, specie, tip și dacă planta este carnivoră. Exemplar Service (8082) folosește b_exemplare, cu două tabele: exemplare (exemplarele individuale ale plantelor, cu zona din grădină) și imagini (imaginile asociate exemplarelor, legate prin foreign key de tabela exemplare cu ștergere în cascadă). User Service (8083) folosește b_users, cu tabela users care stochează username, email, parolă hash, rol, număr de telefon și data creării. Notification Export Service (8084) folosește b_log, cu tabelele export_jobs (istoricul job-urilor de export, cu status PENDING/DONE/FAILED) și notifications (istoricul notificărilor trimise prin EMAIL sau WHATSAPP, cu status și timestamp).

Nu există constrângeri de integritate (foreign keys) între baze de date diferite. Legăturile logice sunt gestionate exclusiv la nivel de aplicație prin REST API: plant_id din exemplare face referire logică la id din plante, iar user_id din b_log face referire logică la id din b_users. Această abordare este specifică arhitecturii de microservicii și evită cuplarea strânsă între servicii.

Separarea bazelor de date aduce mai multe avantaje: independența — o cădere a unei baze nu afectează celelalte; scalabilitatea — fiecare bază poate fi scalată în funcție de încărcarea serviciului corespunzător; securitatea — b_users, care conține parole hash, este izolată de restul; și flexibilitatea tehnologică — fiecare bază poate fi înlocuită independent cu alt sistem (ex: PostgreSQL) fără impact asupra celorlalte servicii.

DESCRIEREA APLICAȚIEI

Aplicația oferă o soluție completă de management pentru o grădină botanică, permițând gestionarea plantelor, exemplarelor, utilizatorilor și generarea de rapoarte statistice.

Arhitectură generală

Sistemul este compus din patru microservicii independente expuse prin API Gateway pe portul 8080:

- **plant-service (8081)** — gestionarea plantelor și statistici
- **exemplar-service (8082)** — gestionarea exemplarelor și imaginilor
- **user-service (8083)** — autentificare JWT și gestionarea utilizatorilor

- **notification-export-service (8084)** — notificări email/WhatsApp și export catalog

4.3. Tipuri de utilizatori

- **Vizitator** — poate vizualiza catalogul de plante, filtra după tip, specie, zonă sau carnivoră, și căuta după denumire sau specie, fără autentificare
- **Angajat** — toate permisiunile vizitatorului, plus operații CRUD pentru plante și exemplare, și export catalog în formatele CSV, JSON, XML și DOC
- **Manager** — toate permisiunile angajatului, plus acces la statistici interactive (3 grafice) și export raport Word cu statistici
- **Administrator** — toate permisiunile vizitatorului, plus CRUD utilizatori, filtrare după rol, export CSV utilizatori, și notificări automate email + WhatsApp la orice modificare a datelor unui utilizator

SABLOANE DE PROIECTARE IMPLEMENTATE

Aplicația implementează șabloane de proiectare din toate cele trei categorii:

- **Factory**(Creațional) — în notification-export-service, pentru crearea dinamică a strategiei de export corespunzătoare formatului solicitat (CSV, JSON, XML sau DOC)
- **Strategy** (Comportamental) — în notification-export-service, fiecare format de export este implementat ca o strategie independentă, interschimbabilă fără modificarea codului client
- **State** (Comportamental) — în notification-export-service, un job de export trece prin stările PENDING → PROCESSING → COMPLETED sau FAILED
- **Observer** (Comportamental) — în user-service, la orice modificare a unui utilizator sunt notificați automat toți observatorii înregistrați: EmailObserver, WhatsAppObserver și AuditObserver
- **Template Method** (Comportamental) — în plant-service, validarea plantelor urmează un șablon fix cu pași precum validateDenumire(), validateSpecie(), validateTip(), cu posibilitatea de a extinde prin hook methods

INTERNAȚIONALIZARE

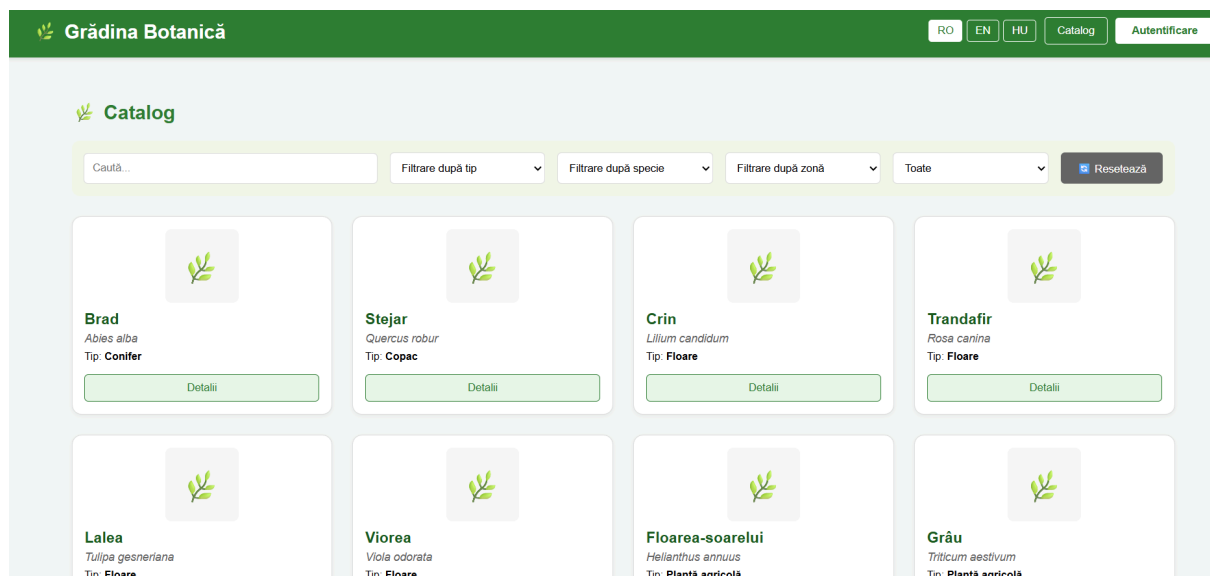
Interfața grafică este disponibilă în trei limbi de circulație internațională — română, engleză și maghiară — implementate cu biblioteca react-i18next. Comutarea între limbi se face dinamic, fără reîncărcarea paginii.

PERSISTENȚĂ DATE

Datele sunt stocate în MySQL 8, organizate în șase tabele principale:

- **plants** — informații despre plante (denumire, specie, tip, carnivoră)
- **exemplare** — exemplare individuale ale plantelor, cu zona din grădină
- **images** — imagini asociate exemplarelor
- **users** — utilizatorii sistemului (username, email, parolă hash, rol, telefon)
- **export_jobs** — istoricul job-urilor de export
- **notifications** — istoricul notificărilor trimise (canal, status, timestamp)

DEMONSTRATIE INTERFATA GRAFICA



Botanikus Kert

ROENHUKatalógusBejelentkezés

Katalógus

Keresés...

Szűrés típus szerint

Szűrés faj szerint

Szűrés zóna szerint

Mind

Reset

Brad

Abies alba

Típus: Conifer

Részletek

Stejar

Quercus robur

Típus: Copac

Részletek

Crin

Lilium candidum

Típus: Floare

Részletek

Trandafir

Rosa canina

Típus: Floare

Részletek

Lalea

Tulipa gesneriana

Típus: Floare

Viorea

Viola odorata

Típus: Floare

Floarea-soarelui

Helianthus annuus

Típus: Plantă agricolă

Grâu

Triticum aestivum

Típus: Plantă agricolă

Botanical Garden

ROENHUCatalogLogin

Catalog

Search...

Filter by type

Filter by species

Filter by zone

All

Reset

Brad

Abies alba

Type: Conifer

Details

Stejar

Quercus robur

Type: Copac

Details

Crin

Lilium candidum

Type: Floare

Details

Trandafir

Rosa canina

Type: Floare

Details

Lalea

Tulipa gesneriana

Type: Floare

Viorea

Viola odorata

Type: Floare

Floarea-soarelui

Helianthus annuus

Type: Plantă agricolă

Grâu

Triticum aestivum

Type: Plantă agricolă

Panou Administrator - Utilizatori						Exportă CSV	+ addUser
Filtrare după tip: Toate							
ID	Utilizator	Email	role	Status	Acțiuni		
1	maaa	maaa@botanic.ro	MANAGER	Active	 		
2	dayna	dmoldovan832@gmail.com	EMPLOYEE	Active	 		
3	ad	ad@botanic.ro	ADMIN	Active	 		
4	adm	adm@botanic.ro	ADMIN	Active	 		

DESCRIERE INTERFATA GRAFICA

1. Prezentare generală

Sistemul permite gestionarea plantelor, a exemplarelor, a utilizatorilor și generarea de rapoarte statistice, având suport pentru trei limbi: română, engleză și maghiară.

2. Structura interfeței

Interfața aplicației este organizată într-o bară de navigare superioară și o zonă principală de conținut.

Bara de navigare conține:

- Titlul aplicației:
 - „Grădina Botanică”
 - click pentru revenirea la pagina principală
- Selector de limbă:
 - RO (română)
 - EN (engleză)
 - HU (maghiară)
- Buton Catalog:
 - acces la pagina principală cu toate plantele
- Buton Panou Manager:
 - vizibil doar pentru utilizatorii cu rol MANAGER

- oferă acces la statistici
 - Buton Panou Administrator:
 - vizibil doar pentru utilizatorii cu rol ADMIN
 - oferă acces la gestionarea utilizatorilor
 - Buton Login / Informații utilizator:
 - autentificare
 - afișarea numelui și rolului utilizatorului curent
-

3. Fluxul de utilizare pentru fiecare tip de utilizator

3.1. Utilizatorul Vizitator (neautentificat)

Vizitatorul poate accesa aplicația fără autentificare și poate utiliza următoarele funcționalități.

Catalogul de plante

Pagina principală afișează toate plantele din grădina botanică, sortate după:

- tip
- specie

Fiecare plantă este afișată sub forma unui card care conține:

- 1–3 imagini reprezentative
- denumirea populară
- specia
- tipul plantei
- indicator pentru plante carnivore

Căutarea și filtrarea

Vizitatorul poate utiliza:

- căutare text după denumire sau specie
- filtrare după tip
- filtrare după specie
- filtrare după zonă
- filtrare după plante carnivore:
 - Toate

- Carnivore
- Non-carnivore

Există și un buton:

- „Resetare filtre”

Filtrarea se realizează în timp real.

Vizualizarea detaliilor unui exemplar

Apăsând butonul „Detalii”:

- utilizatorul este redirecționat către pagina plantei
 - sunt afișate toate exemplarele asociate
 - fiecare exemplar poate avea:
 - zonă
 - imagini asociate
-

3.2. Utilizatorul Angajat (autentificat)

Angajatul are acces la toate funcționalitățile vizitatorului, plus funcționalități suplimentare.

Adăugarea unei plante noi

Buton:

- „+ Adaugă Plantă”

Formularul conține:

- denumire populară
- specie
- tip
- checkbox pentru plantă carnivoră

După salvare:

- planta este stocată în baza de date
- apare automat în catalog

Editarea unei plante

Pe fiecare card sunt disponibile:

- Editează
- Șterge

Ștergerea este permisă doar dacă planta nu are exemplare asociate.

Gestionarea exemplarelor

În pagina de detalii:

- adăugare exemplar
- editare exemplar
- ștergere exemplar
- încărcare imagini JPG/PNG

Ștergerea unui exemplar elimină automat și imaginile asociate.

Exportul catalogului

Angajatul poate exporta catalogul în:

- CSV
- JSON
- XML
- DOC

Fișierul este generat și descărcat automat.

3.3. Utilizatorul Manager (autentificat)

Managerul are toate funcționalitățile angajatului și acces la statistici.

Panoul de statistici

Buton suplimentar:

- „Statistici”

Pagina conține trei grafice:

1. Distribuția plantelor pe tipuri

- grafic tip bară
- afișează numărul plantelor pentru fiecare tip

2. Carnivore vs Necarnivore

- grafic tip donut
- afișează proporțiile și procentele

3. Top specii

- grafic bară orizontală
- primele 5 specii cu cele mai multe plante

Sunt afișate și informații suplimentare:

- total plante
- număr tipuri distincte
- specia dominantă

Exportul statisticilor în Word

Buton:

- „Export Word”

Raportul generat conține:

- titlu
 - data generării
 - grafice integrate
 - tabele statistice
-

3.4. Utilizatorul Administrator (autentificat)

Administratorul gestionează utilizatorii sistemului.

Panoul Administrator

Buton:

- „Panou Administrator”

Pagina afișează lista utilizatorilor.

Funcționalități disponibile

Administratorul poate:

- vizualiza utilizatori
- filtra după rol
- adăuga utilizatori
- edita utilizatori
- șterge utilizatori

Tabel utilizatori

Coloane:

- ID
- Username
- Email
- Rol
- Status
- Număr telefon
- Acțiuni

Adăugare utilizator

Formular:

- username
- email
- parolă
- rol
- număr telefon

Editare utilizator

Administratorul poate:

- modifica datele
- trimite mesaj personalizat utilizatorului

Export utilizatori CSV

Buton:

- „Export CSV”

Fișierul poate fi deschis în:

- Excel
- LibreOffice Calc

Notificări automate

La modificarea datelor:

- email prin SMTP
- notificare WhatsApp simulată

Administratorul poate adăuga un mesaj personalizat în notificări.

4. Procesul de autentificare

Pentru acces la funcționalitățile restricționate este necesară autentificarea.

Pașii autentificării

1. Apăsare buton „Login Personal”
2. Introducere username și parolă
3. Apăsare buton „Intră în cont”

După autentificare

- token JWT stocat în localStorage
- afișarea utilizatorului și rolului
- apariția butoanelor specifice rolului
- disponibilitate logout

Crearea conturilor

Conturile sunt create exclusiv de administrator.

Vizitatorii nu își pot crea singuri cont.

5. Schimbarea limbii

Aplicația suportă:

- română
- engleză
- maghiară

Schimbarea limbii se face prin:

- RO
- EN
- HU

Comutarea este instantanee datorită bibliotecii [react-i18next](#).

Sunt traduse:

- butoanele
 - mesajele
 - graficele
 - tabelele
-

6. Gestionarea erorilor și feedback-ul utilizatorului

Acțiune	Feedback
Autentificare reușită	Redirecționare la catalog
Autentificare eșuată	„Invalid credentials”
Adăugare plantă	„Plantă adăugată!”
Ștergere plantă cu exemplare	„Planta are exemplare asociate”
Export reușit	„Exportul a fost generat cu succes!”
Export eșuat	„Eroare la generarea exportului”

Notificări trimise	„Notifications sent!”
Încărcare imagine	„Se încarcă...”
Date în curs de încărcare	„Se încarcă...”

DIAGrame

DESCRIERE

DESCRIEREA DIAGrameLOR UML

1. Diagrama de Cazuri de Utilizare (Use Case Diagram)

Diagrama de cazuri de utilizare prezintă interacțiunile dintre actorii sistemului și funcționalitățile oferite.

Actorii identificați

Actor	Descriere
Vizitator	Utilizator neautentificat, poate doar vizualiza catalogul
Angajat	Personalul grădinii, gestionează plante și exemplare
Manager	Personal cu rol decizional, acces la statistici și rapoarte
Administrator	Gestionează utilizatorii și permisiunile sistemului

Categorii de cazuri de utilizare

- Vizualizare și căutare:
 - vizualizarea catalogului de plante, sortat după tip și specie
 - căutarea după denumire sau specie
 - filtrarea după tip, specie, zonă și plante carnivore
- Gestionare plante:

- operații CRUD pentru plante
- validare la inserare și actualizare
- ștergere cu verificarea existenței exemplarelor asociate
- Gestionare exemplare:
 - operații CRUD pentru exemplare
 - încărcare și ștergere de imagini
 - filtrarea exemplarelor după plantă sau zonă
- Export:
 - exportul catalogului de plante în formatele CSV, JSON, XML și DOC (angajat și manager)
 - exportul statisticilor în Word (manager)
 - exportul listei de utilizatori în CSV (administrator)
- Statistici:
 - distribuția plantelor pe tipuri
 - compararea plantelor carnivore și necarnivore
 - topul speciilor după numărul de plante
- Gestionare utilizatori:
 - operații CRUD pentru utilizatori
 - filtrarea listei după rol
 - notificarea utilizatorilor prin email și WhatsApp la modificarea datelor de autentificare

2. Diagrame de Secvență

Diagramele de secvență ilustrează fluxurile de interacțiune dintre obiecte pe parcursul execuției unei funcționalități.

Procese de autentificare și utilizatori

- autentificare utilizator:
 - login
 - generare token JWT
 - stocare în localStorage
- CRUD utilizatori:
 - adăugare
 - modificare
 - ștergere
 - vizualizare utilizatori
- filtrare utilizatori după rol

Procese de gestionare a plantelor

- vizualizare catalog:
 - încărcarea și afișarea plantelor sortate
- căutare și filtrare:
 - filtrare după tip
 - filtrare după specie
 - filtrare după zonă
 - filtrare după plante carnivore
- CRUD plante:
 - adăugare
 - modificare
 - ștergere
 - validare și verificarea exemplarelor asociate

Procese de gestionare a exemplarelor

- CRUD exemplare:
 - adăugare
 - modificare
 - ștergere
- gestionare imagini:
 - încărcare imagini
 - vizualizare imagini
 - ștergere imagini

Procese de export și notificări

- export catalog:
 - generare CSV
 - generare JSON
 - generare XML
 - generare DOC
- export statistici Word
- export utilizatori CSV
- notificări:
 - trimiterea notificărilor la modificarea datelor utilizatorului

Procese de statistici

- vizualizare statistici:
 - încărcarea și afișarea graficelor

3. Diagrame de Clase

Diagramele de clase prezintă structura statică a sistemului, incluzând clasele principale, atributele, metodele și relațiile dintre acestea.

Plant Service (8081)

Gestionează entitatea **Planta**:

- denumire
- specie
- tip
- carnivoră

Include implementarea șablonului Template Method prin:

- **PlantValidator**
- **StandardPlantValidator**

Serviciul de statistici calculează:

- distribuția plantelor pe tipuri
- raportul carnivore/necarnivore
- topul speciilor

ChartImageService generează imaginile PNG pentru grafice folosind JFreeChart.

Exemplar Service (8082)

Gestionează:

- entitatea **Exemplar**
- entitatea **Imagine**

Exemplar este asociat unei plante prin **plantId**.

Relația dintre **Exemplar** și **Imagine** este one-to-many, iar ștergerea unui exemplar determină ștergerea în cascadă a imaginilor asociate.

User Service (8083)

Gestionează entitatea **User**:

- username
- email
- parolă hash
- rol

- stare activă
- număr de telefon

Implementează:

- Observer Pattern
- Command Pattern

Observatorii notifică utilizatorii prin email și WhatsApp la modificarea contului.

Export & Notification Service (8084)

Implementează:

- Factory
- Strategy
- State

Strategy este utilizat pentru:

- CSV
- JSON
- XML
- DOC

State gestionează stările:

- PENDING
- PROCESSING
- COMPLETED
- FAILED

Gestionează entitățile:

- **ExportJob**
- **Notification**

4. Diagrame ER (Entitate–Relație)

Diagramele ER descriu structura bazelor de date și relațiile dintre tabele.

Sistemul utilizează patru baze de date separate, fiecare corespunzând unui microserviciu.

Bază de date	Serviciu	Tabele
plants_db	Plant Service	plante
exemplars_db	Exemplar Service	exemplare, imagini
users_db	User Service	users
export_notification_db	Export & Notification Service	export_jobs, notifications

Relații între tabele

- În **exemplars_db**, tabelele **exemplare** și **imagini** sunt conectate printr-o relație one-to-many:
 - **exemplare.id** → **imagini.exemplar_id**
 - constrângere **ON DELETE CASCADE**
- Nu există constrângeri de integritate între baze de date diferite.
- Legăturile logice sunt gestionate prin REST API la nivel de aplicație.

5. Diagramă de Componente

Diagrama de componente ilustrează arhitectura generală a sistemului și interacțiunea dintre componente.

Componente principale

- Frontend (React):
 - interfață grafică
 - suport pentru română, engleză și maghiară
 - comunică exclusiv cu API Gateway
- API Gateway (Spring Cloud Gateway):
 - punct unic de intrare
 - rutare cereri
 - validare JWT
 - politici CORS
- Plant Service:
 - gestionarea plantelor
 - statistici

- Exemplar Service:
 - gestionarea exemplarelor
 - gestionarea imaginilor
- User Service:
 - autentificare JWT
 - gestionarea utilizatorilor
- Export & Notification Service:
 - export catalog
 - export statistici
 - notificări

Relațiile dintre componente

- Frontend-ul trimite cereri HTTP către API Gateway.
- Gateway-ul validează token-ul JWT și redirecționează cererile către microserviciul corespunzător.
- Microserviciile comunică între ele prin REST API folosind [RestTemplate](#).
- Export & Notification Service preia date de la Plant Service și User Service pentru generarea exporturilor și notificărilor.

6. Diagramă de Deployment

Diagrama de deployment prezintă distribuția componentelor software pe infrastructura hardware.

Mediul de dezvoltare

Serviciu	Port	Descriere
Frontend	3000	Aplicația React
API Gateway	8080	Spring Cloud Gateway
Plant Service	8081	Gestionare plante
Exemplar Service	8082	Gestionare exemplare

User Service	8083	Gestionare utilizatori
Export & Notification Service	8084	Export și notificări

Bazele de date

Fiecare microserviciu utilizează propria bază de date MySQL:

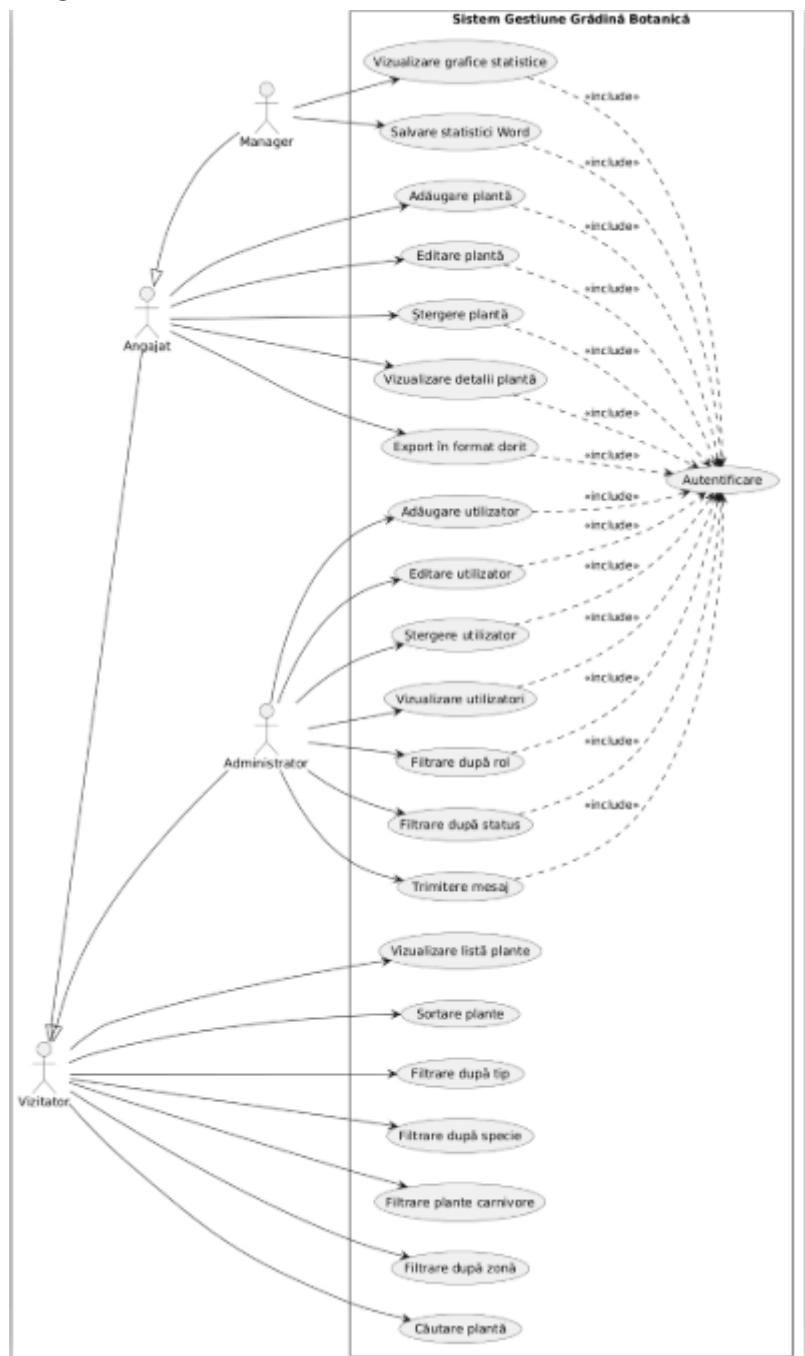
- plants_db
- exemplars_db
- users_db
- export_notification_db

Toate rulează pe portul 3306, dar folosesc baze de date separate.

Comunicare externă

- Notification Service trimite email-uri prin SMTP (Gmail/Mailtrap)
- Notification Service trimite notificări WhatsApp prin Twilio API (simulat în lipsa cheilor de acces)

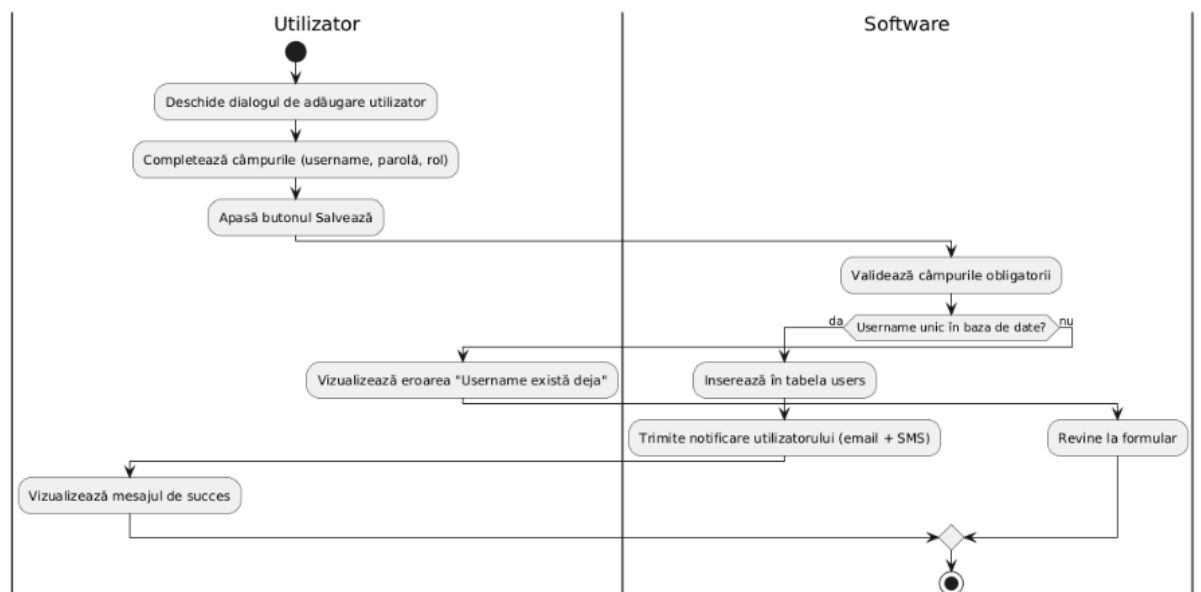
Diagrama cazurilor de utilizare



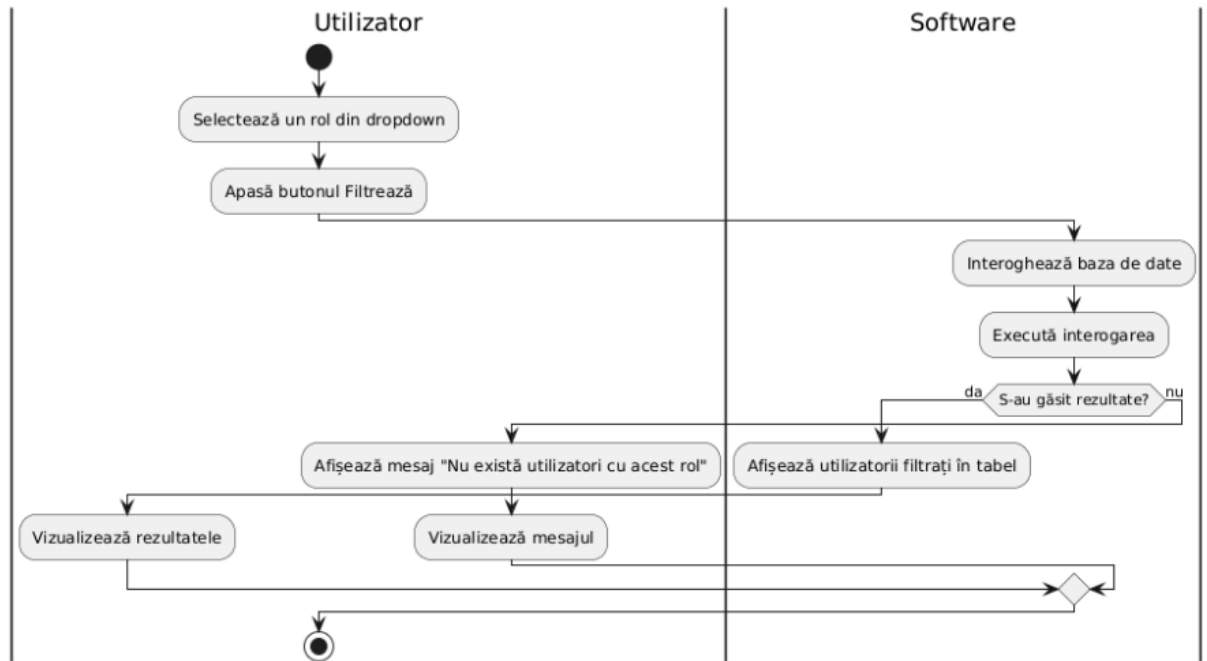
- Diagramele de activități
 - adaugare exemplar



- adaugare utilizator

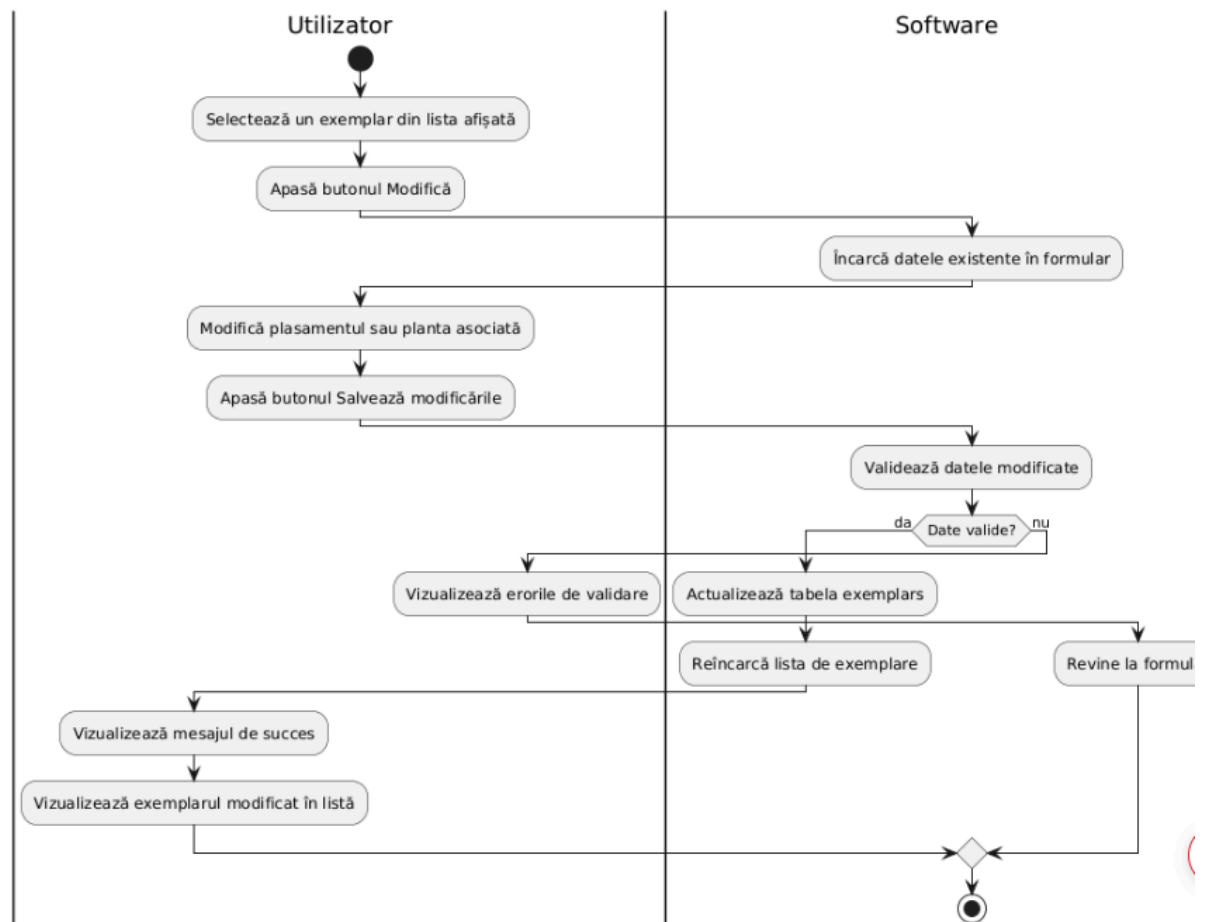


- filtrare utilizator

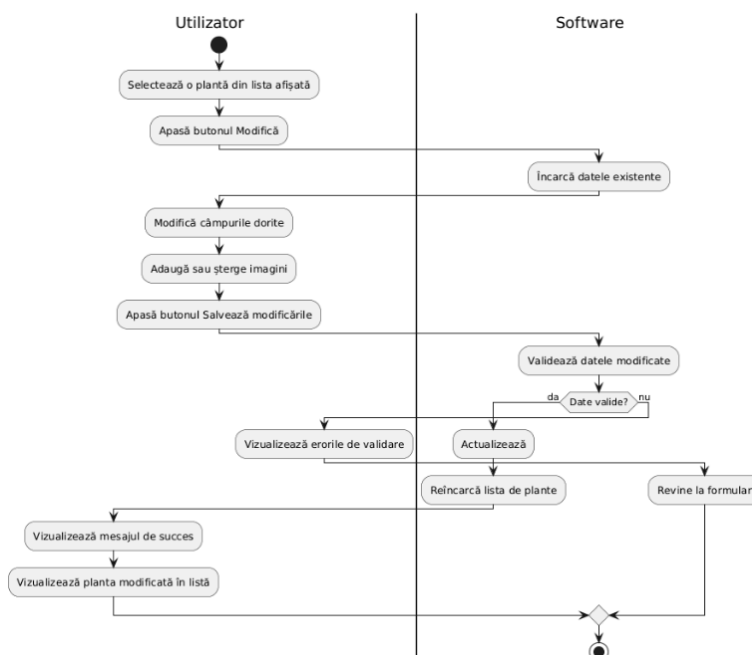


-

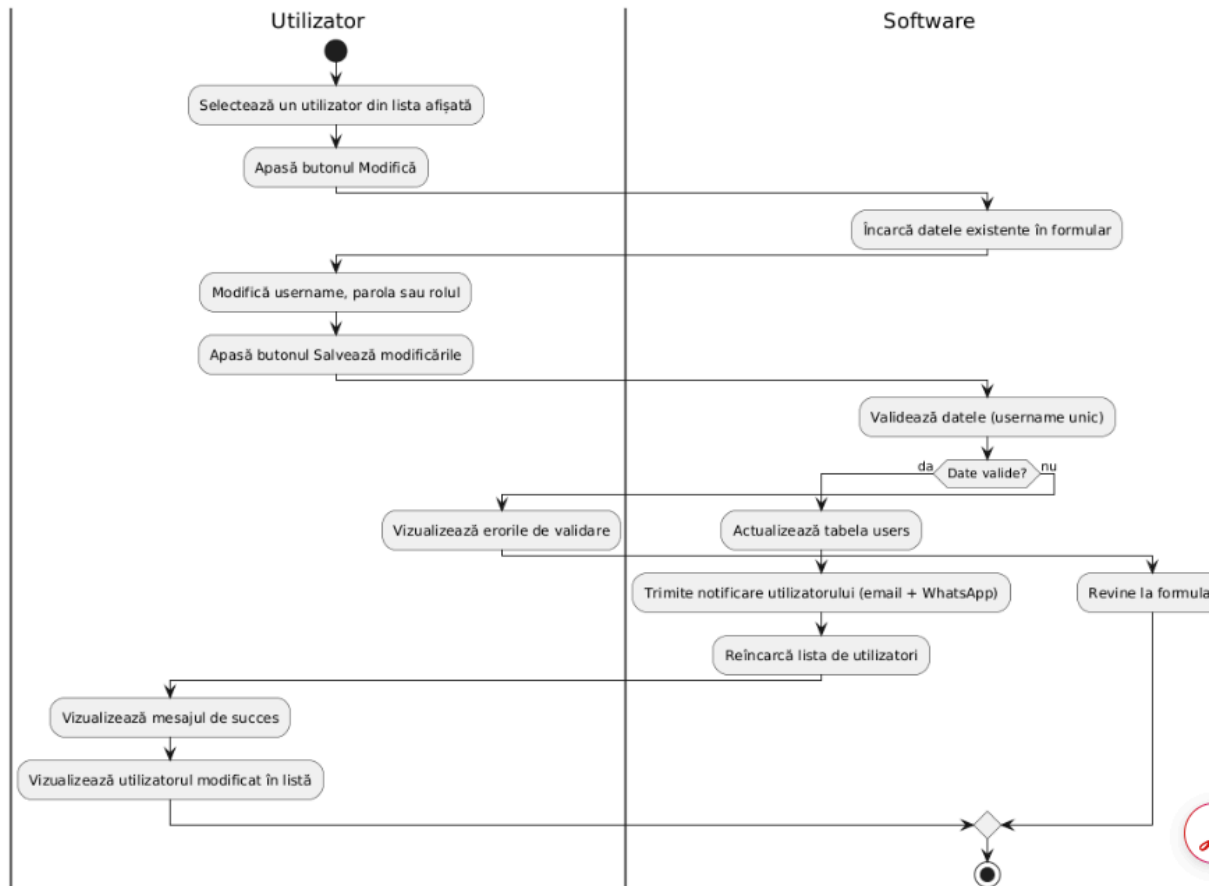
- modificare exemplar



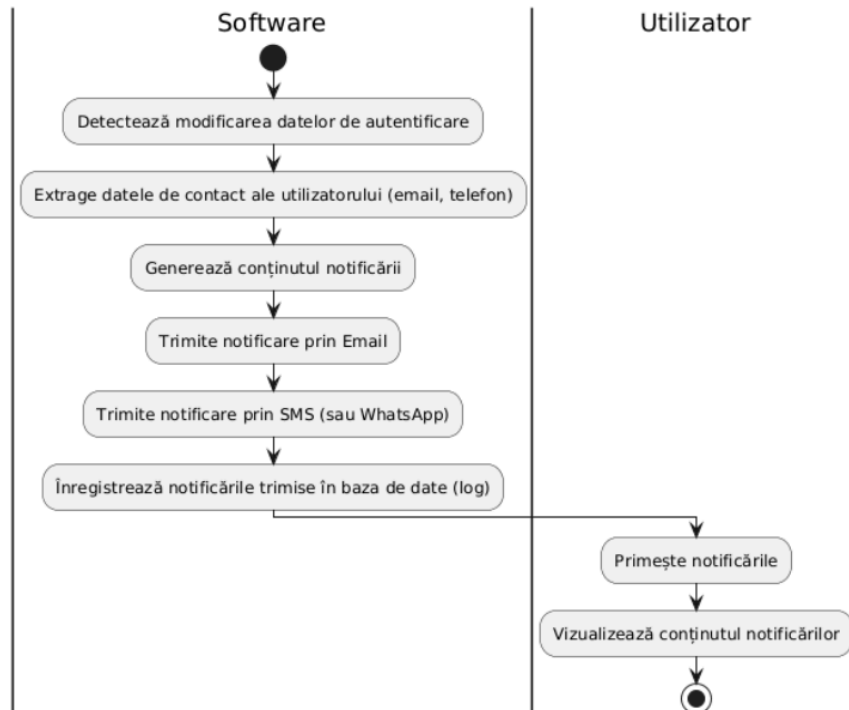
- modificare planta



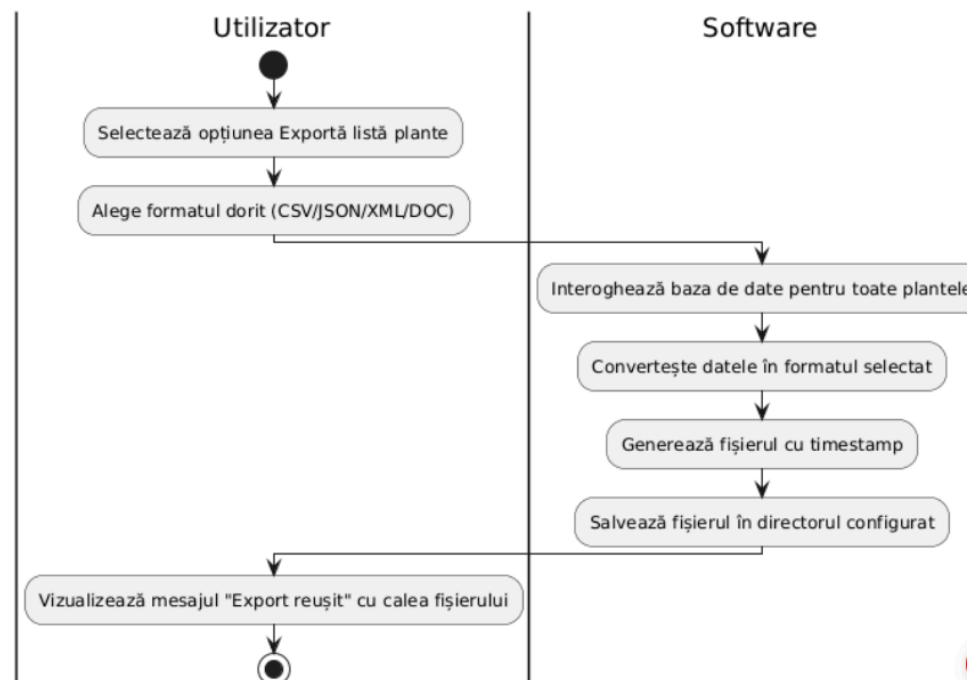
- modificare utilizator



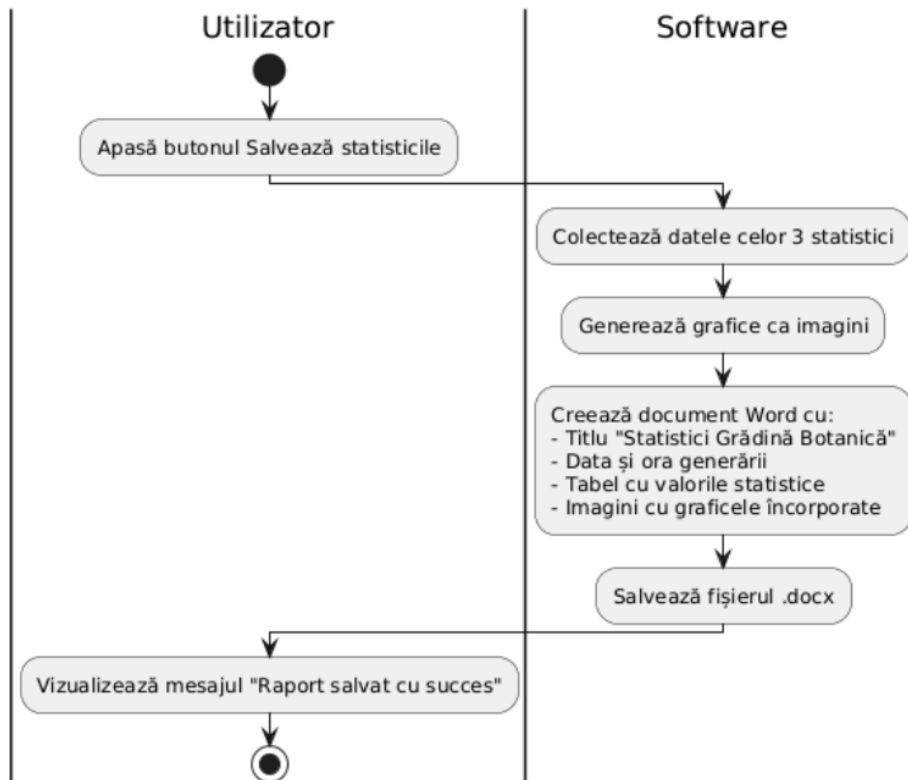
- notificare utilizator modificare date



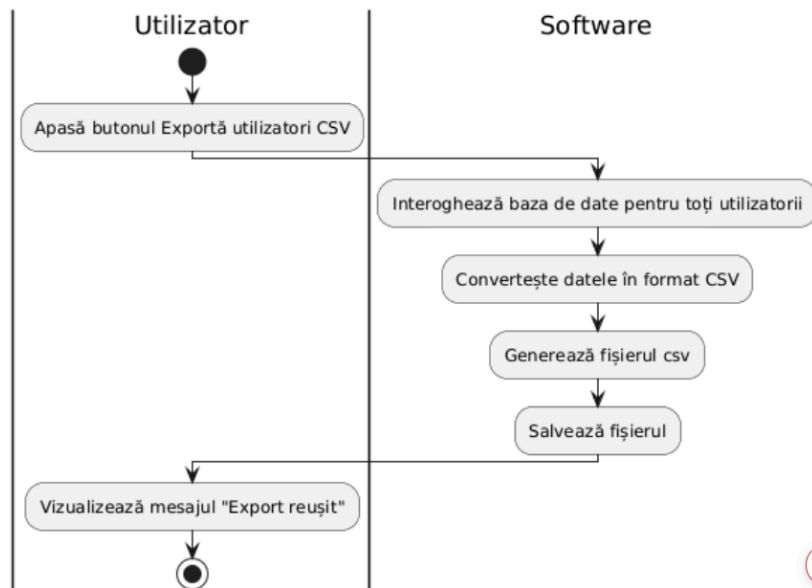
- salvare format



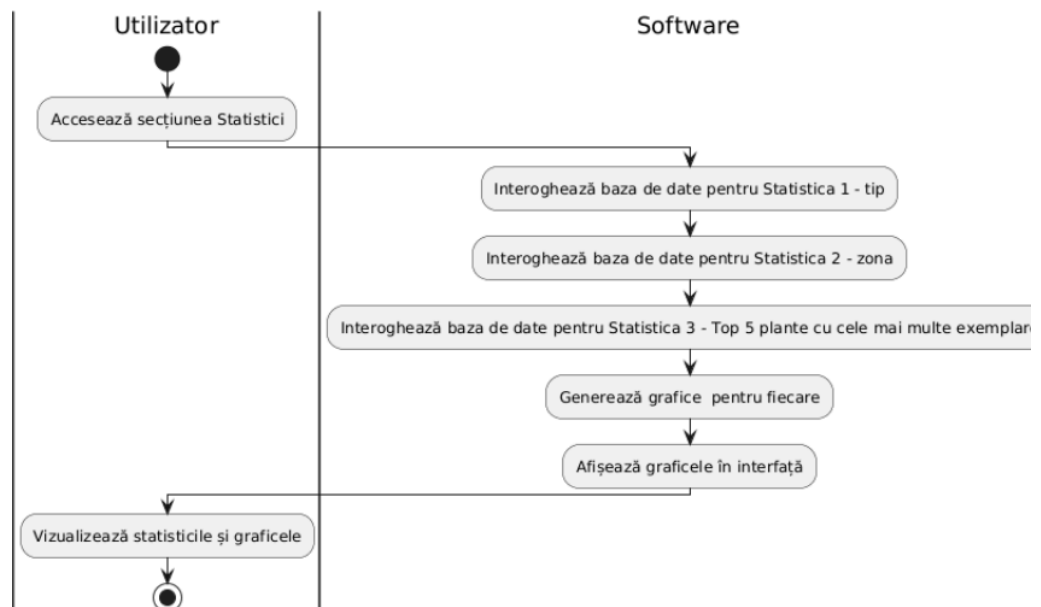
- salvare statistici



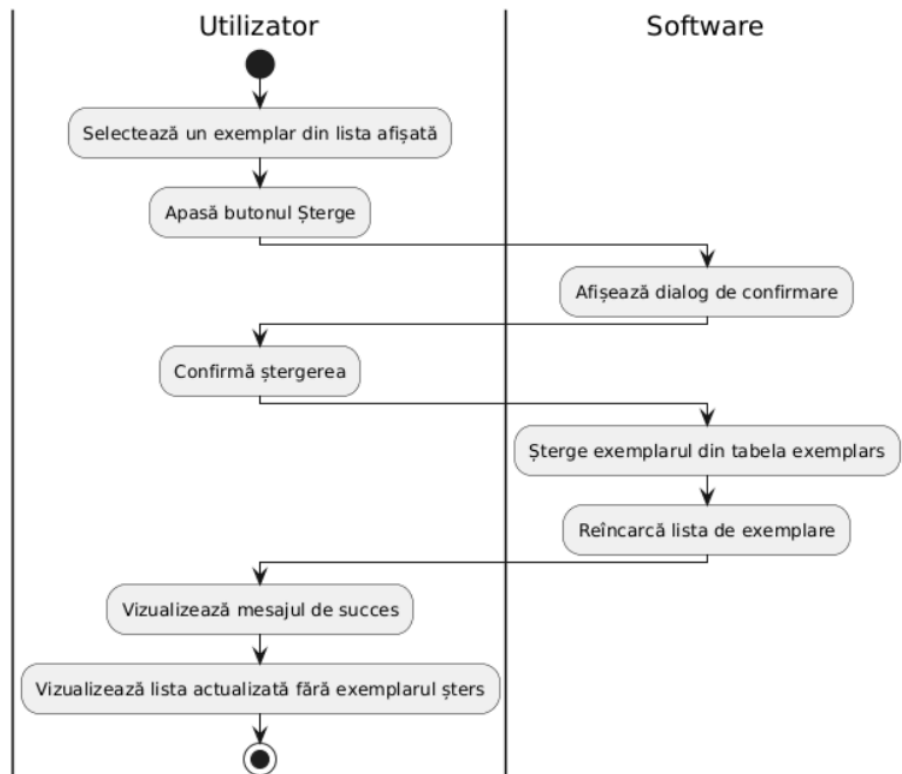
- salvare utilizator



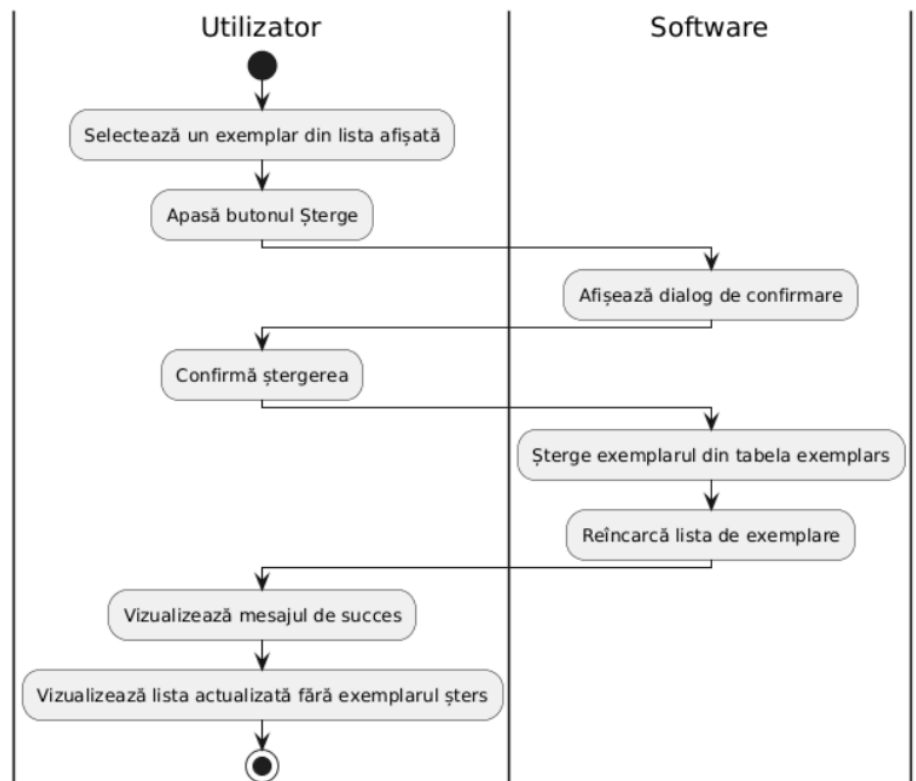
- statistici



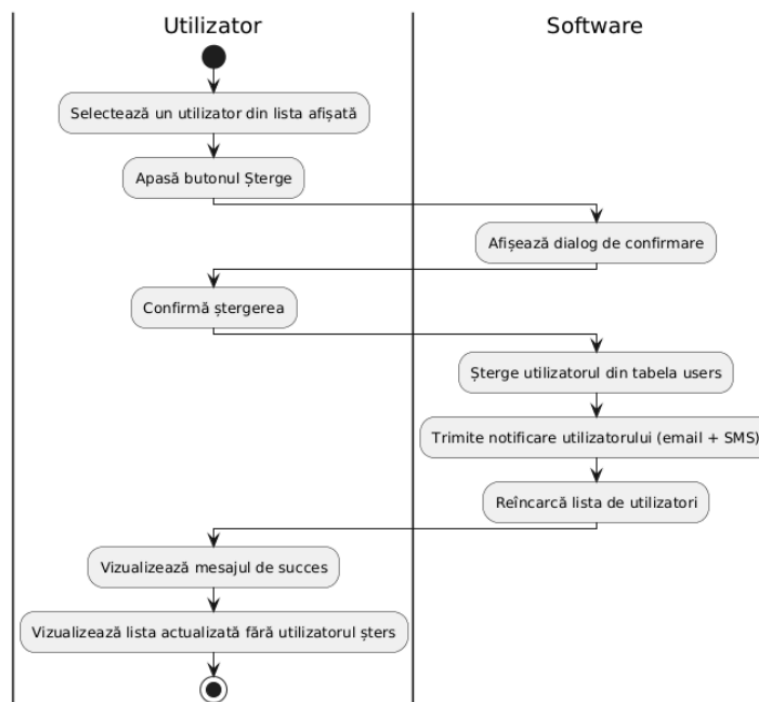
- stergere exemplar



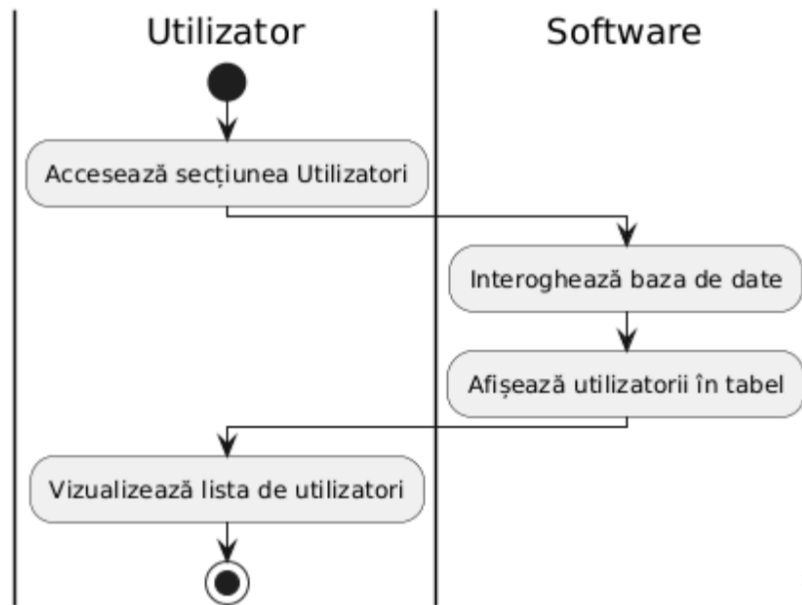
- stergere planta



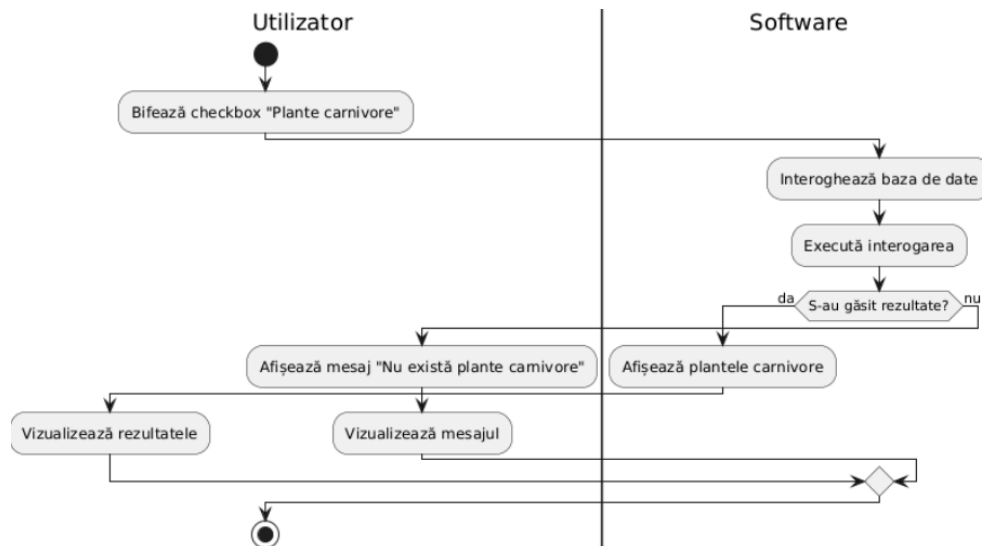
- stergere utilizator



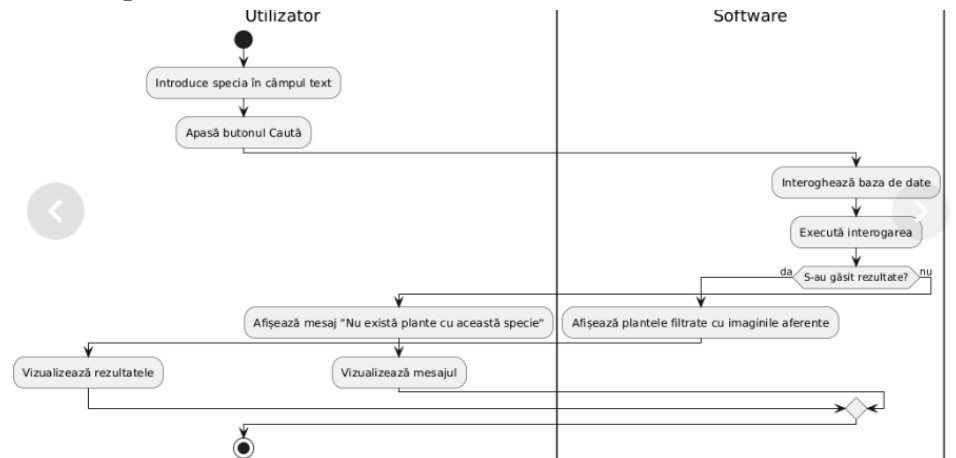
- vizualizare utilizator



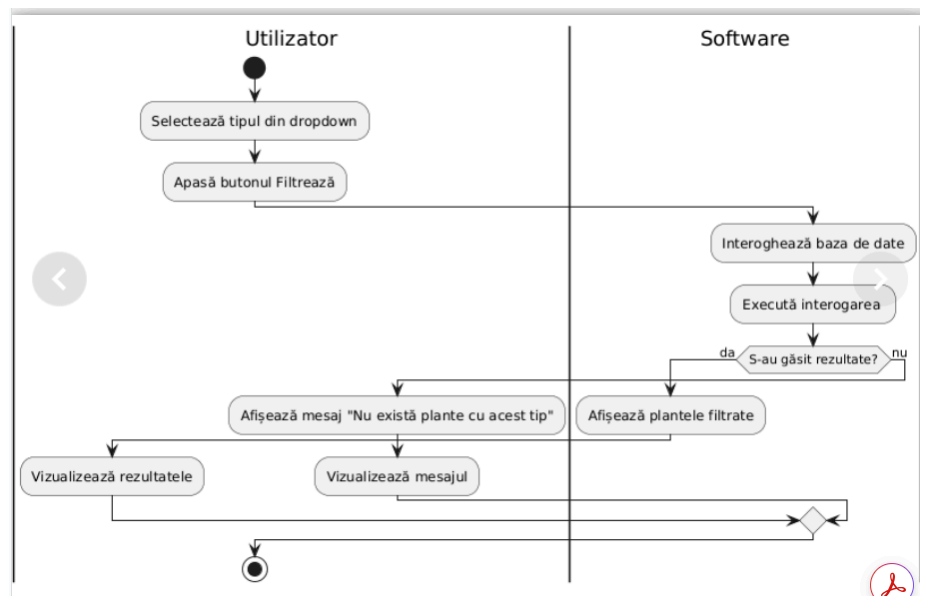
- filtrare carnivore



- filtrare specie



- filtrare tip



```

sequenceDiagram
    participant U as Utilizator
    participant S as Software

    U->>S: Selectează zona din dropdown
    S->>U: 
    U->>S: Apasă butonul Filtrează
    S->>S: Interoghează baza de date
    S->>S: Execută interogarea
    S->>S: S-au găsit rezultate?
    S-->>S: da
    S-->>S: nu
    S->>U: Afişează mesaj "Nu există plante în această zonă"
    S->>U: Afişează plantele care au exemplare în zona respectivă
    U->>U: Vizualizează rezultatele
    U->>U: Vizualizează mesajul
    U->>U: 
    
```

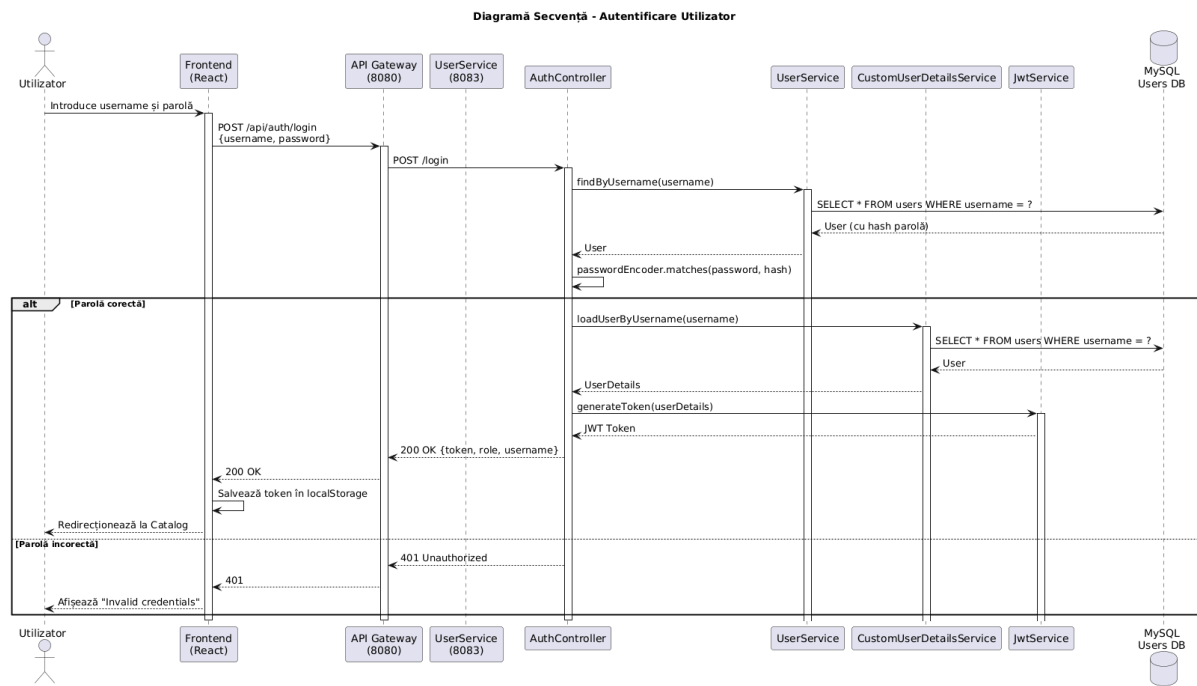
The diagram illustrates the 'Verificare zona' use case. It begins with the 'Utilizator' (User) selecting a zone from a dropdown menu and pressing the 'Filtrează' (Filter) button. This triggers the 'Software' to query the database and execute the search. The software then checks if results were found. If 'da' (yes), it displays the plants with samples in the respective zone to the user. If 'nu' (no), it displays a message stating 'Nu există plante în această zonă' (No plants exist in this zone). The user then visualizes the results or the message, and the process ends.

[illegible]

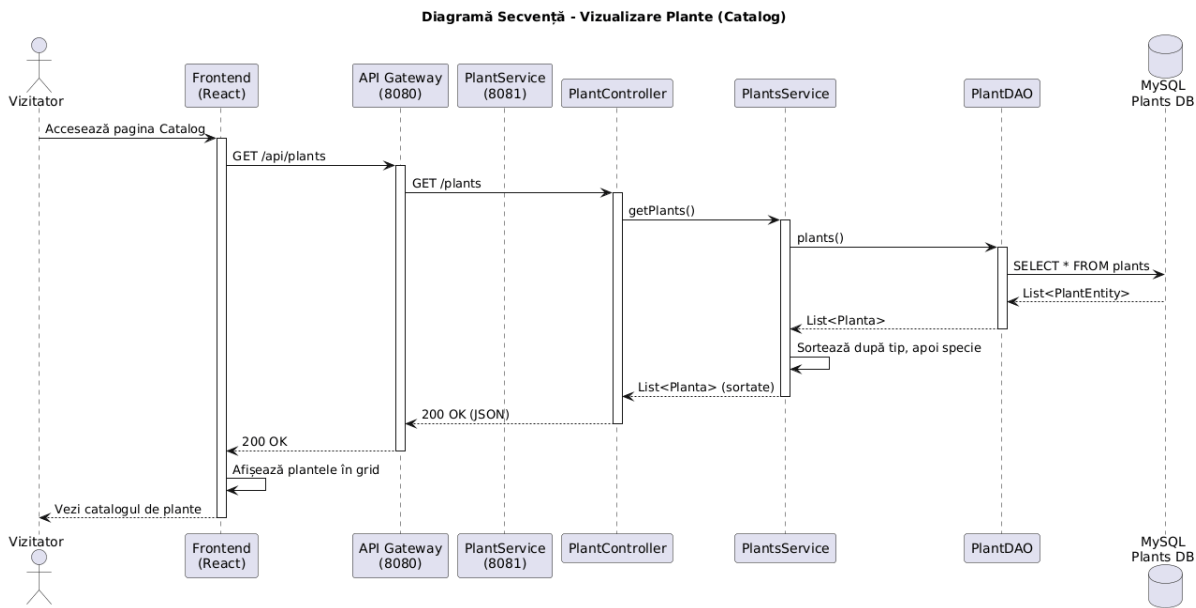
```
graph TD; U(( )) --> A1[Accesează pagina principală]; A1 --> A2[Interoghează baza de date]; A2 --> A3[Sortează rezultatele după type, apoi după species]; A3 --> A4[Afișează plantele]; A4 --> A5[Vizualizează lista de plante cu imagini]; A5 --> U2(( ))
```

DIAGrame DE SECVENTE

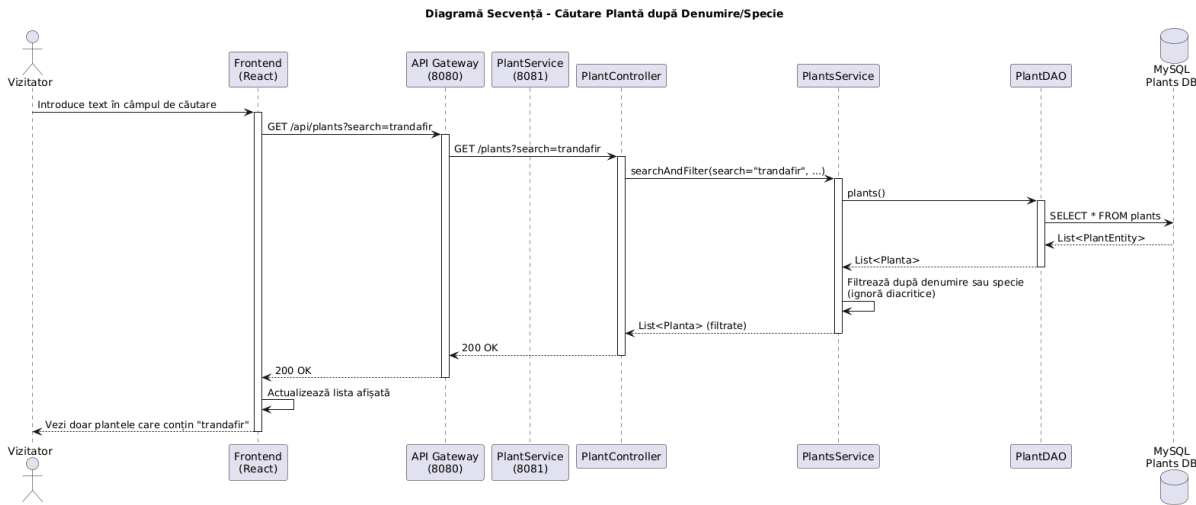
AUTENTIFICARE



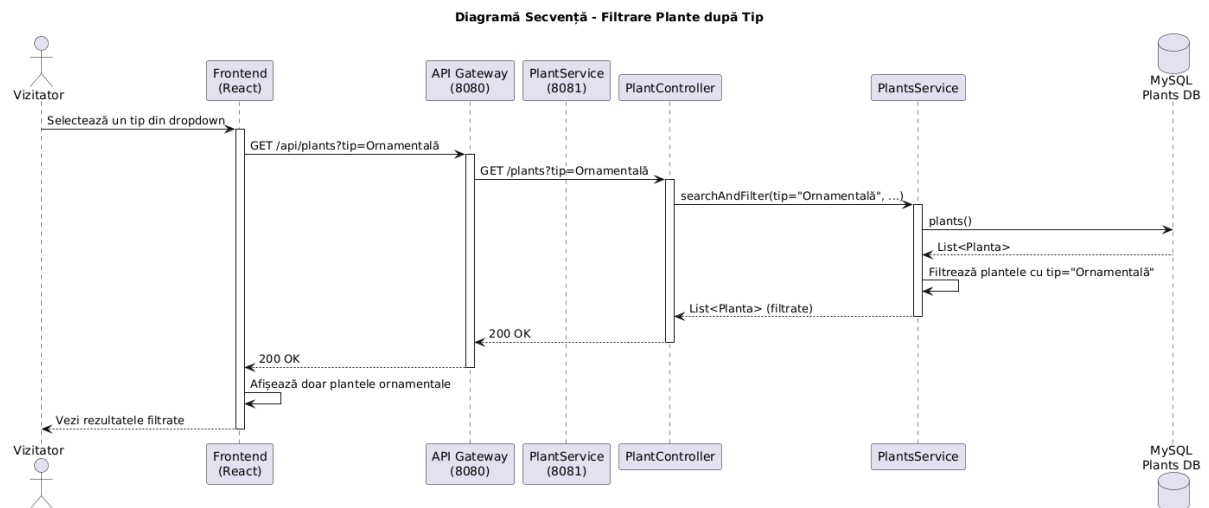
VIZUALIZARE PLANTE



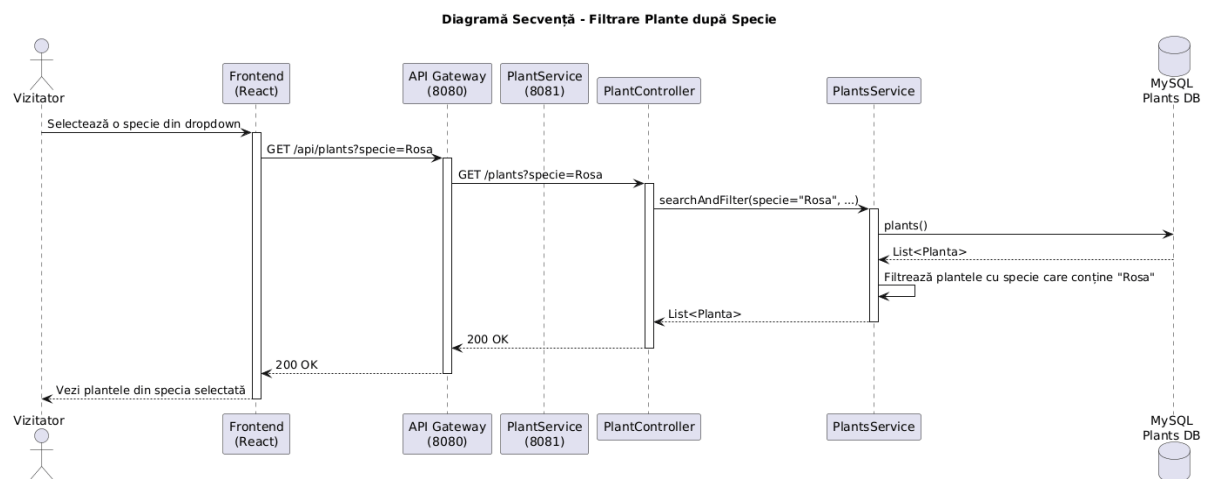
CAUTARE PLANTA



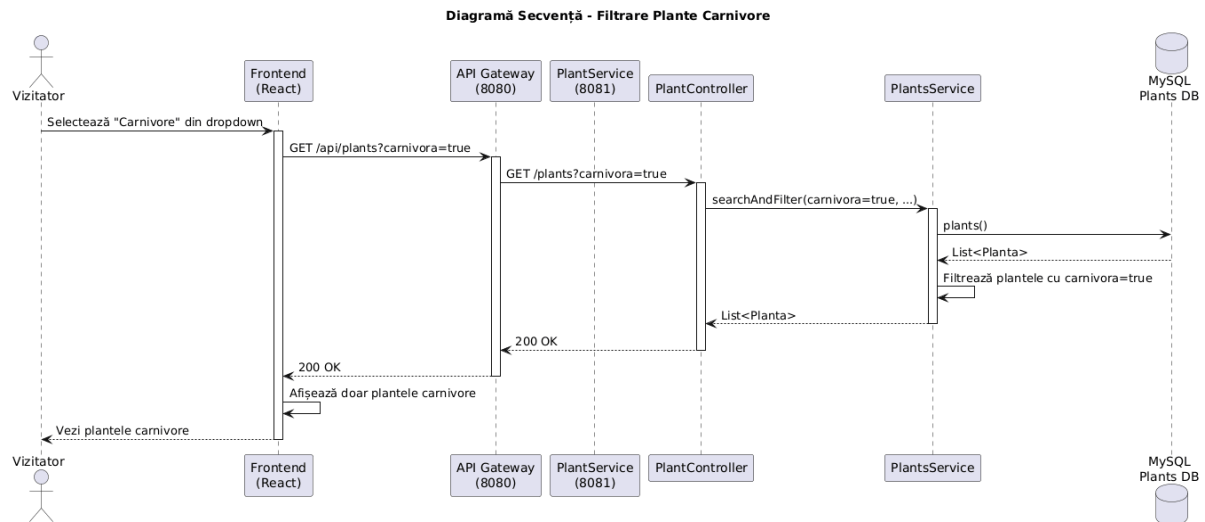
FILTRARE TIP



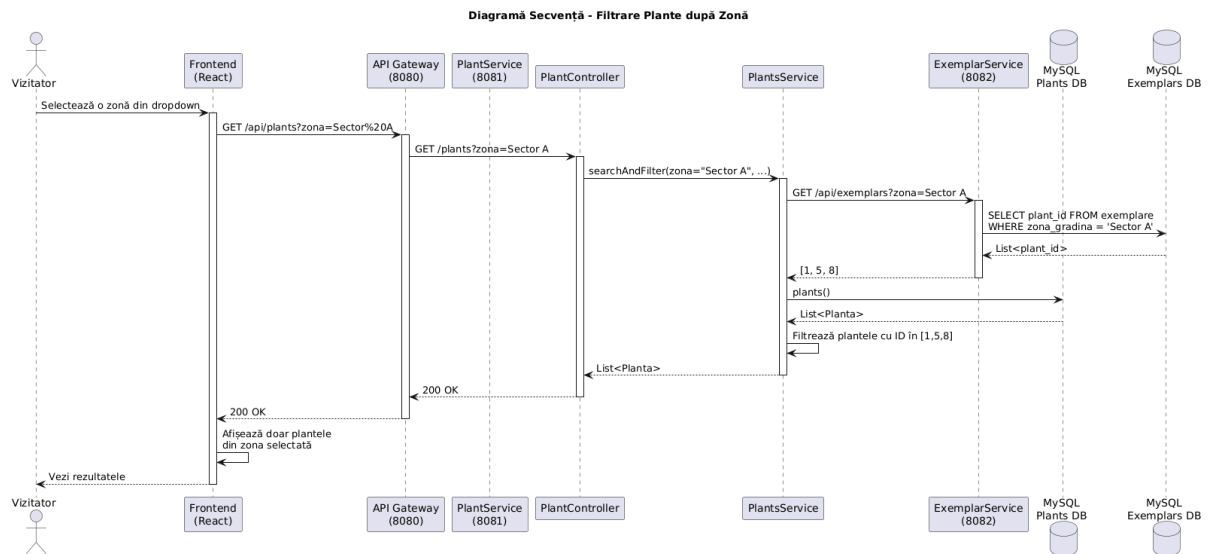
FILTRARE SPECIE



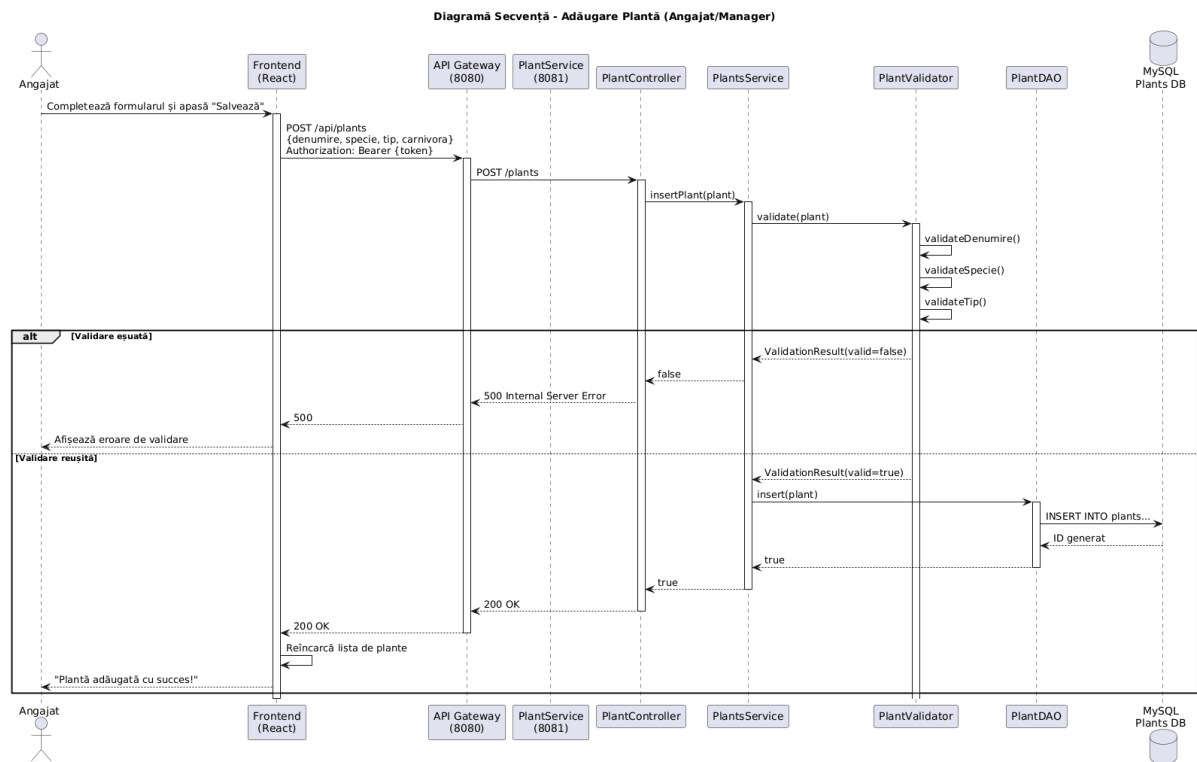
FILTRARE CARNIVORE



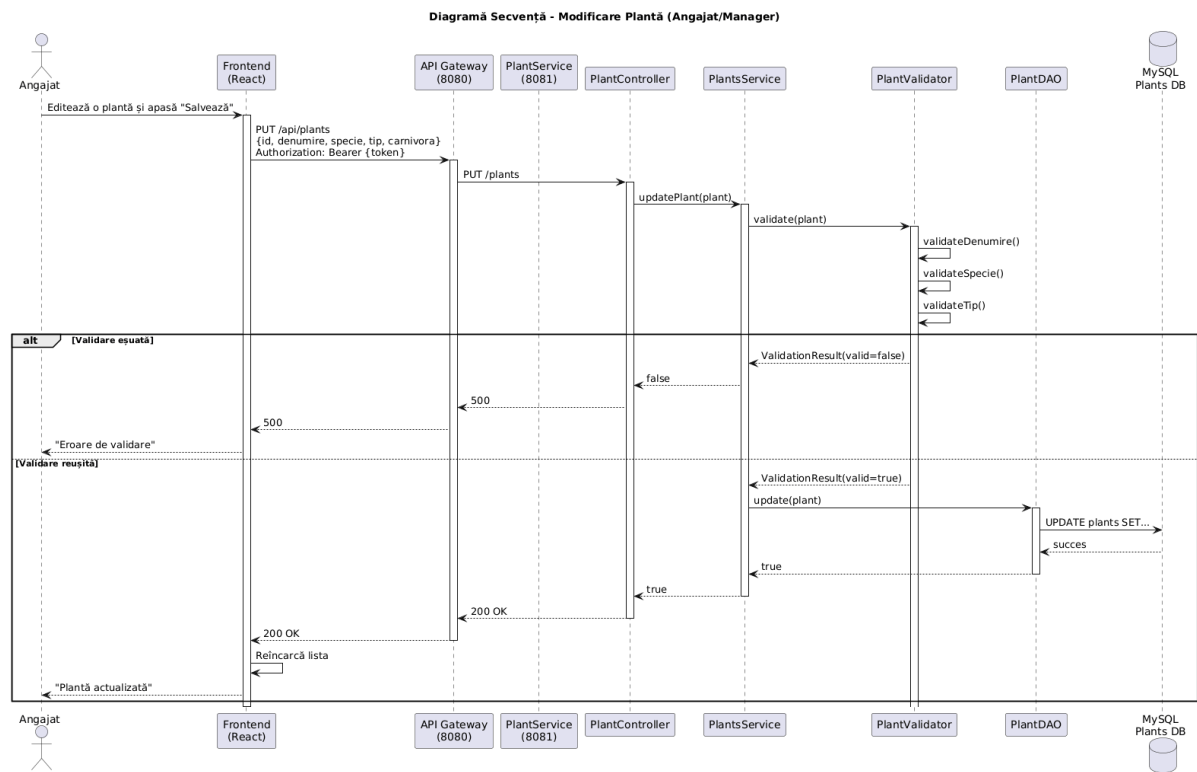
FILTRARE ZONA



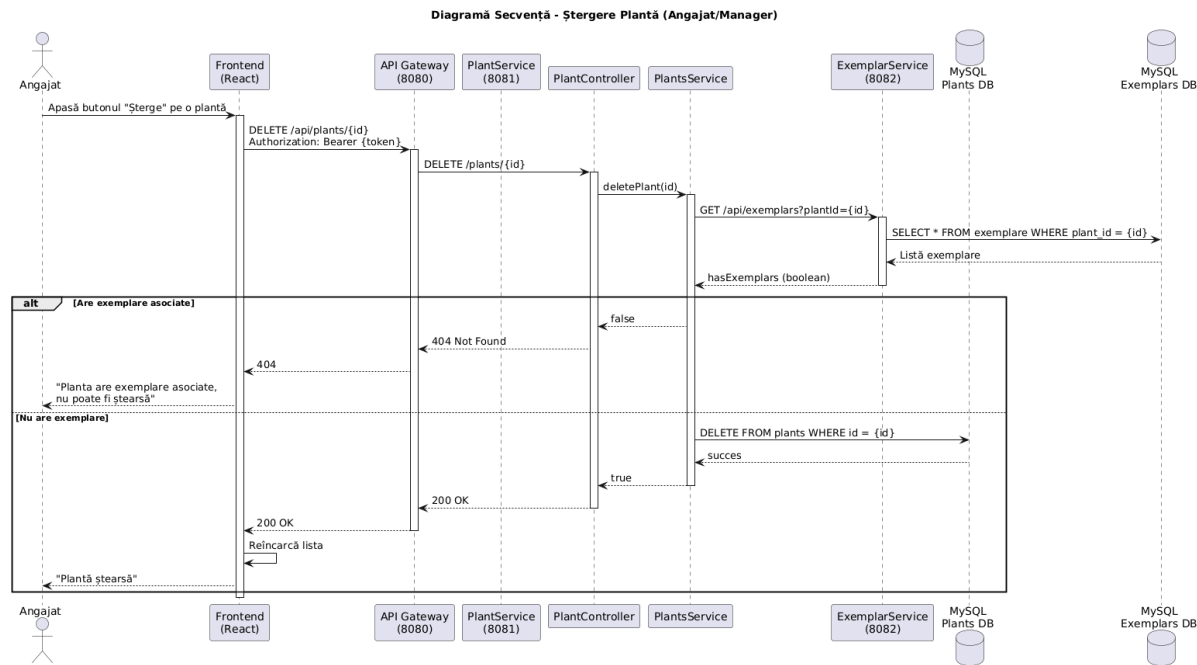
ADAUGARE PLANTA



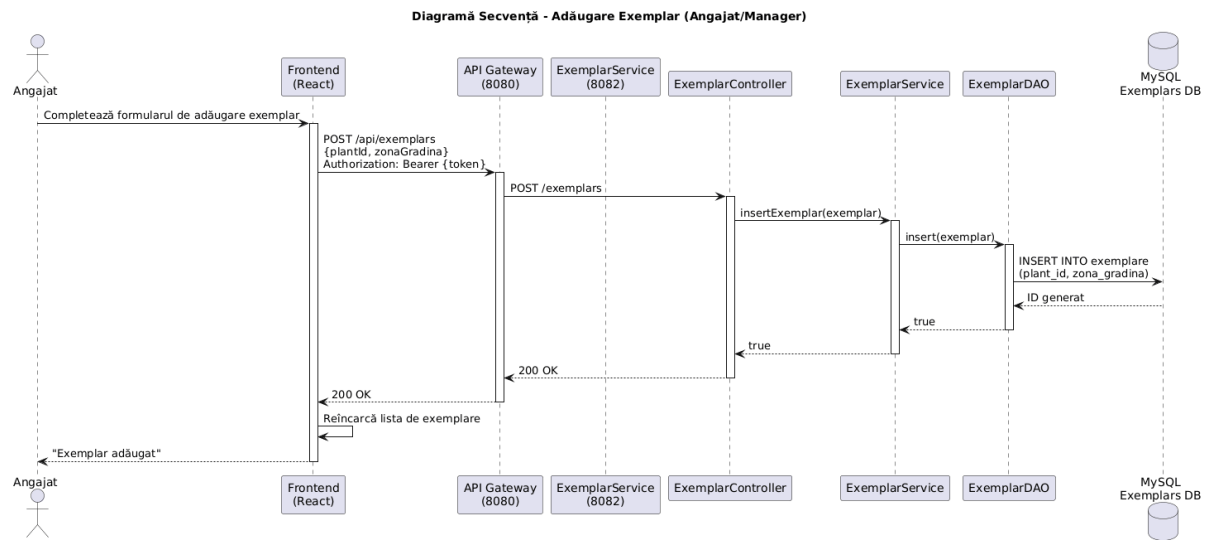
EDITARE PLANTA



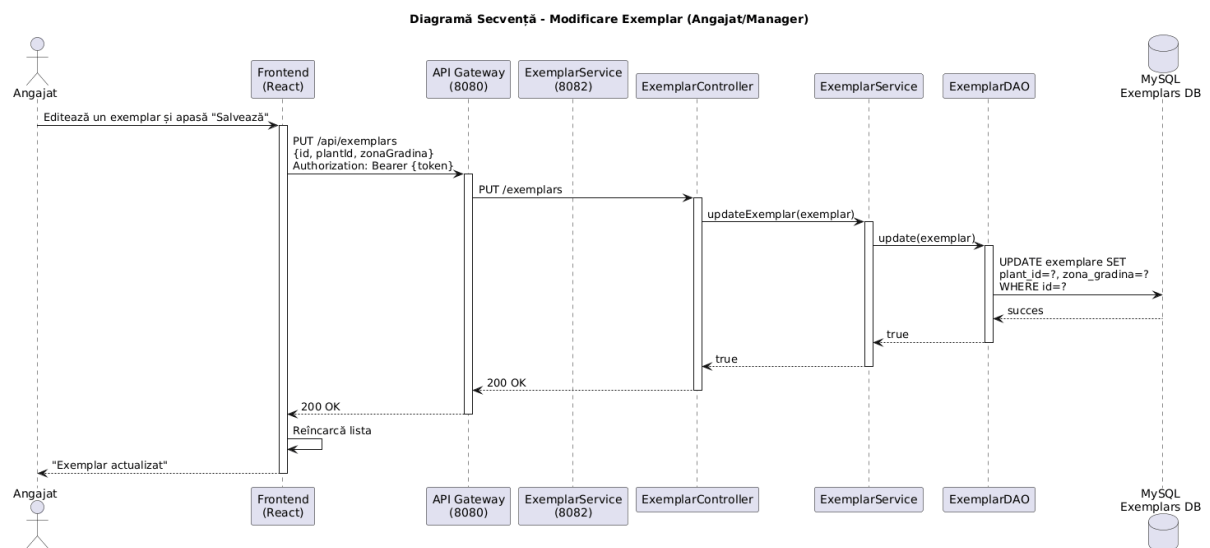
STERGERE PLANTA



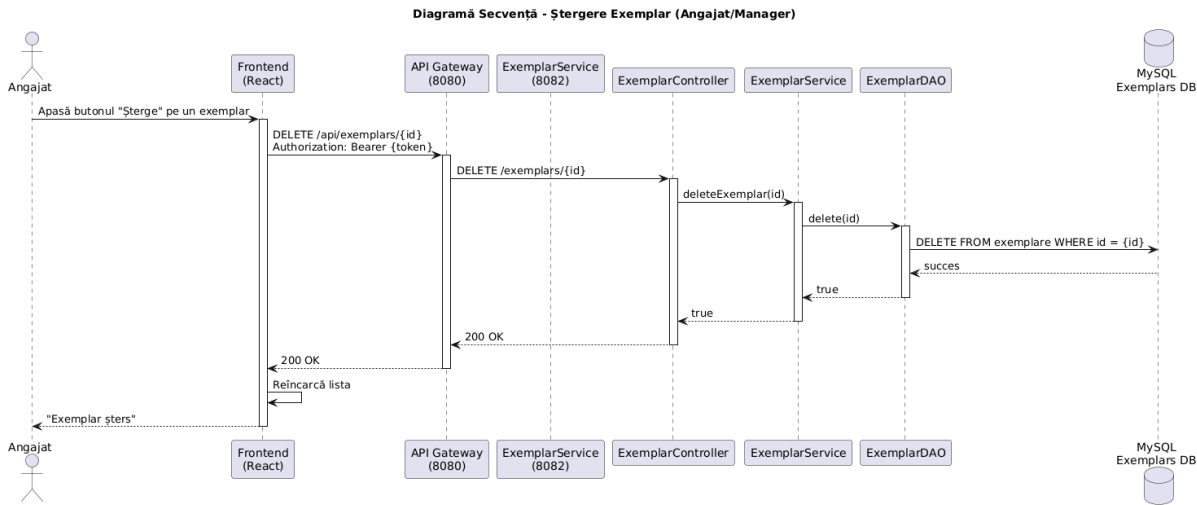
ADĂUGARE EXEMPLAR



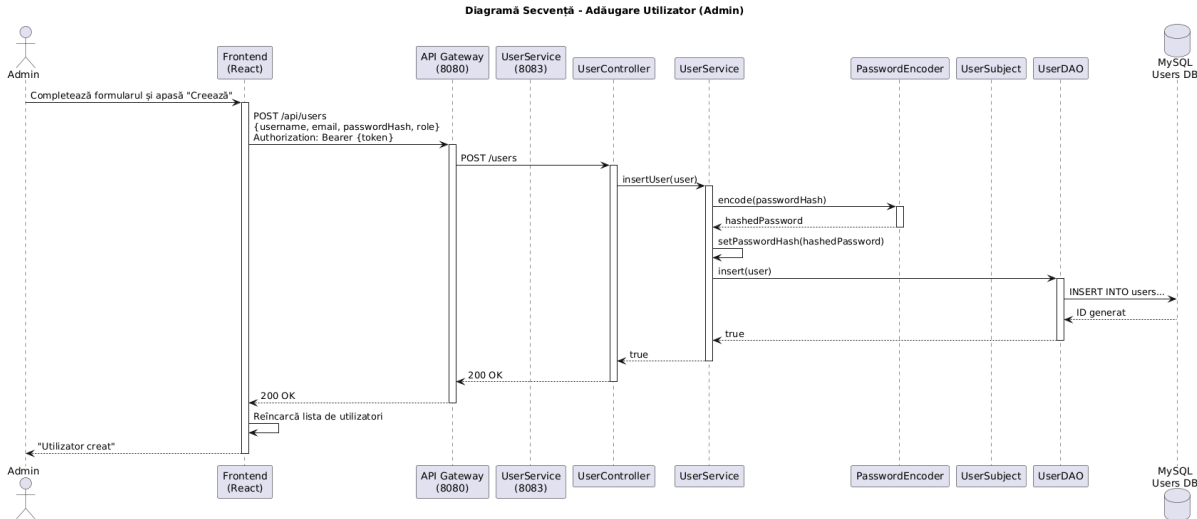
EDITARE EXEMPLAR



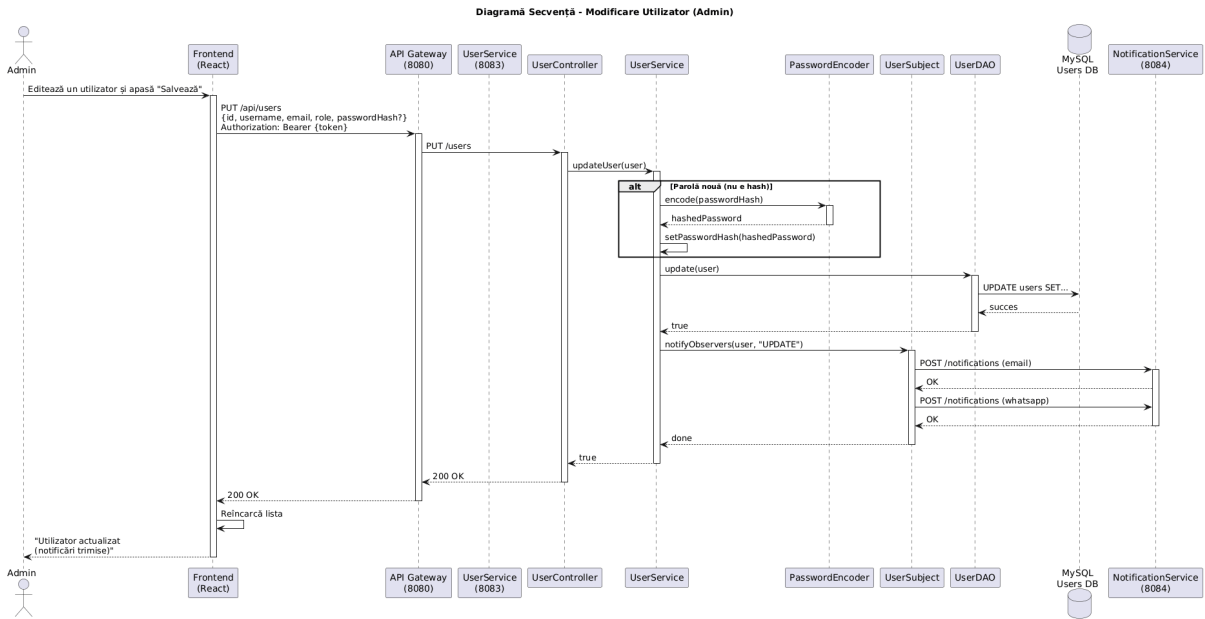
STERGERE EXEMPLAR



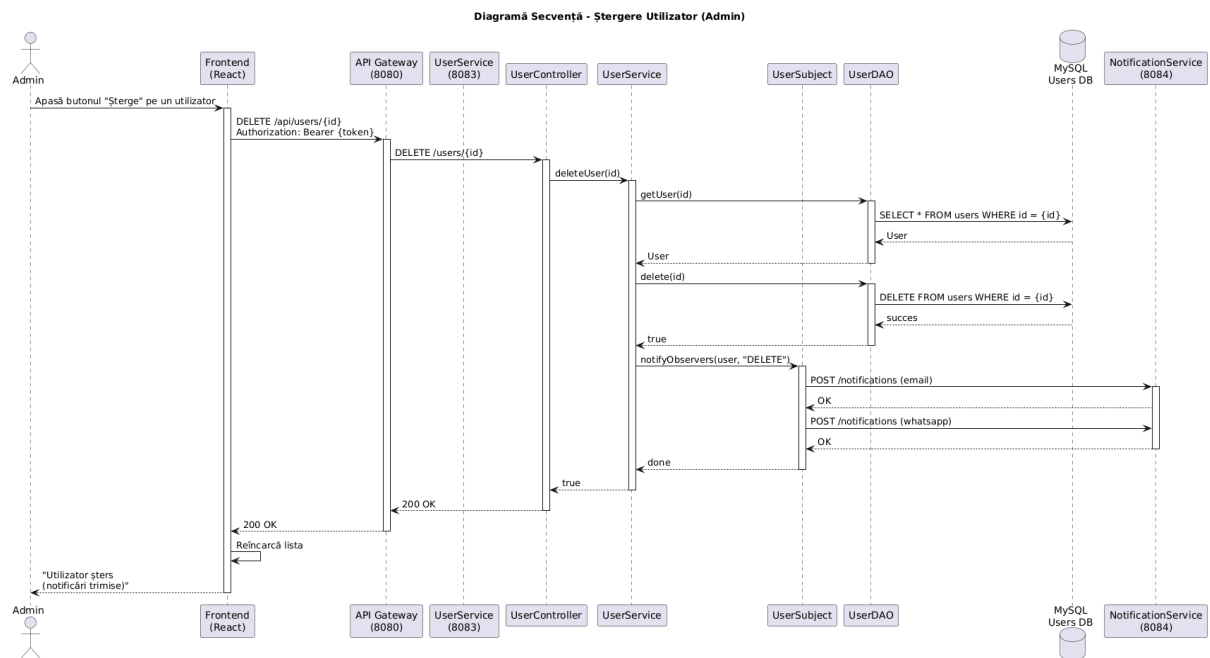
ADAUGARE UTILIZATOR



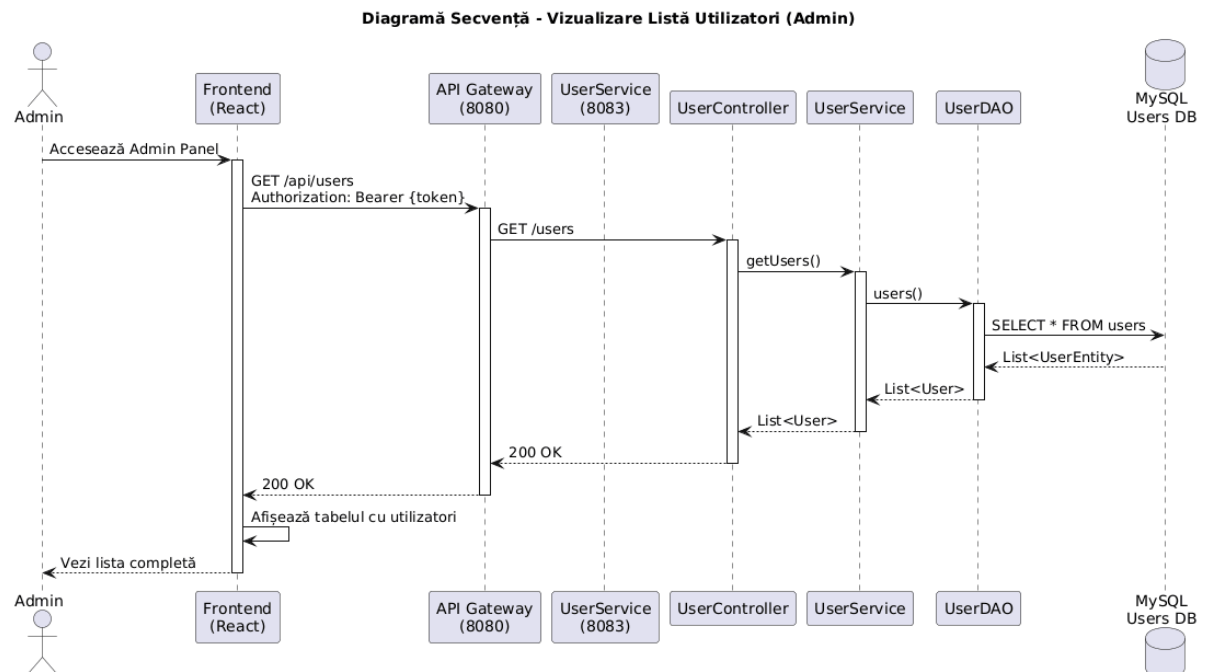
EDITARE UTILIZATOR



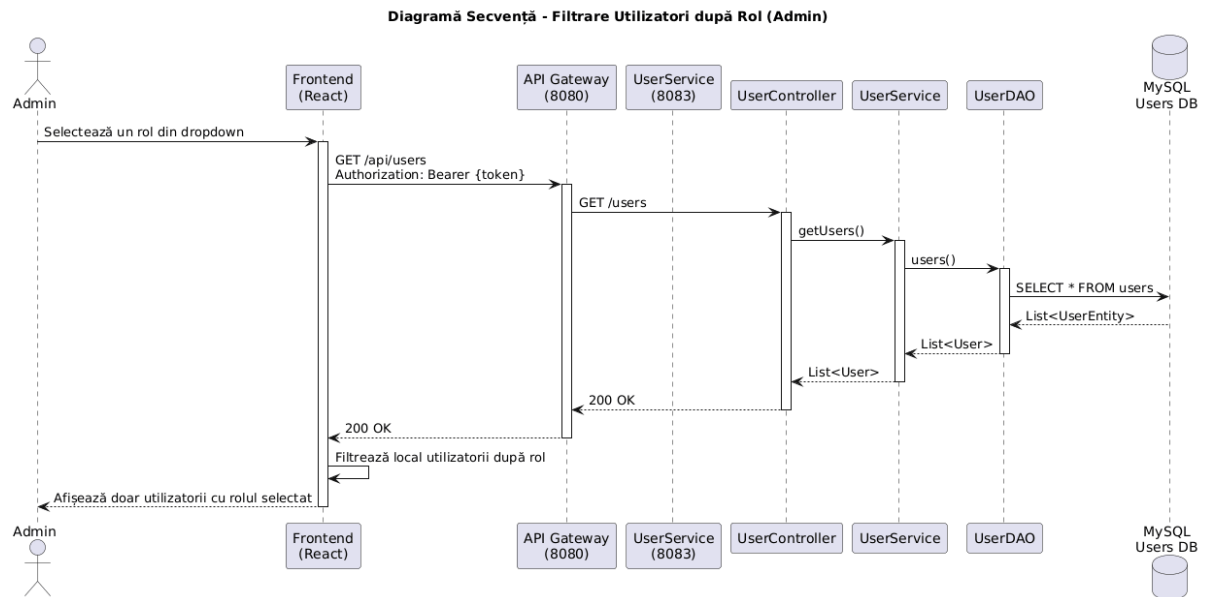
STERGERE UTILIZATOR



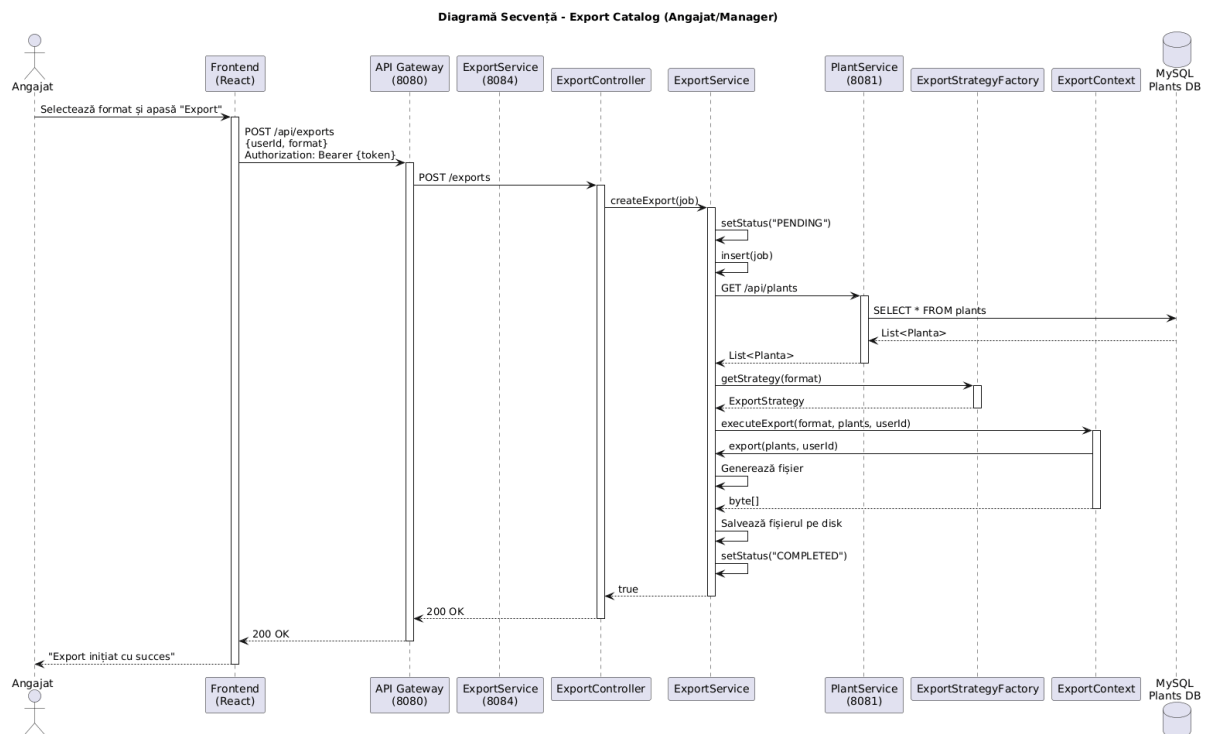
VIZUALIZARE UTILIZATORI



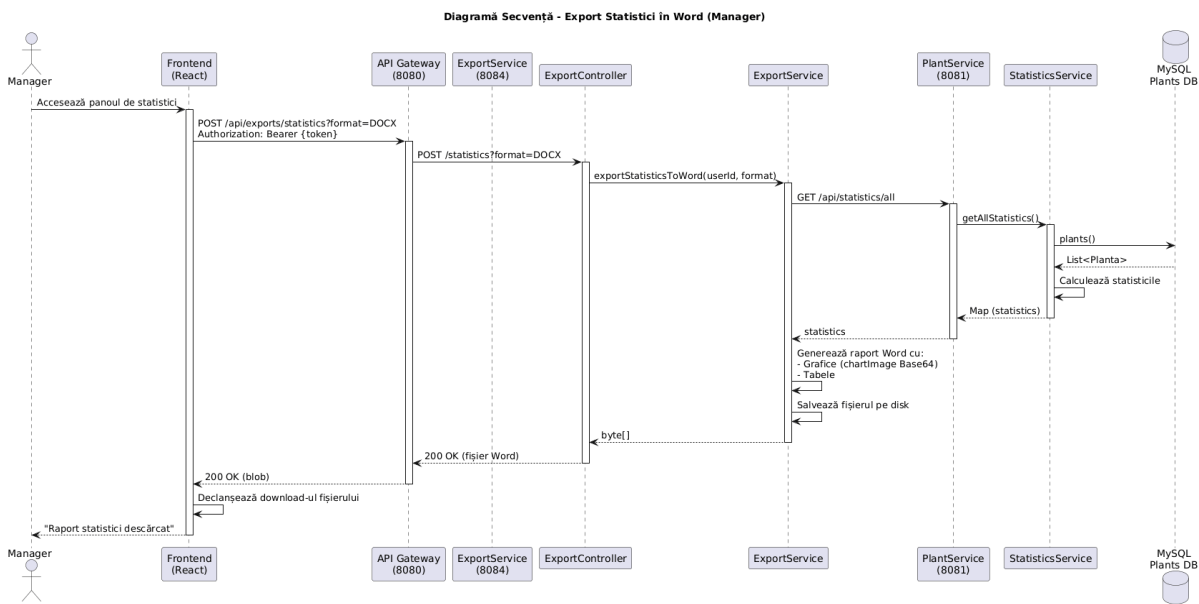
FILTRARE UTILIZATORI ROL



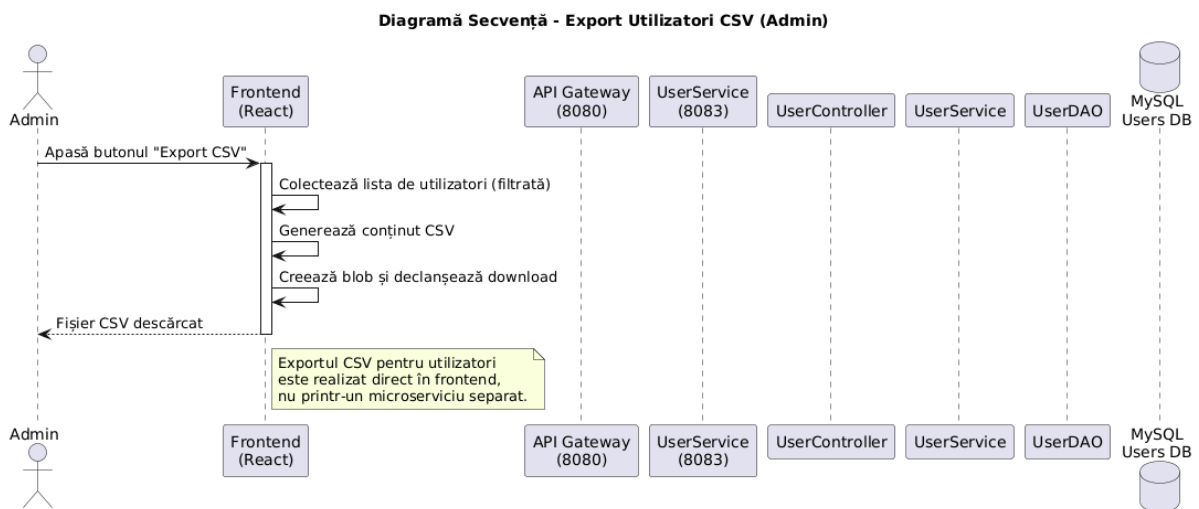
SALVARE EXPORT



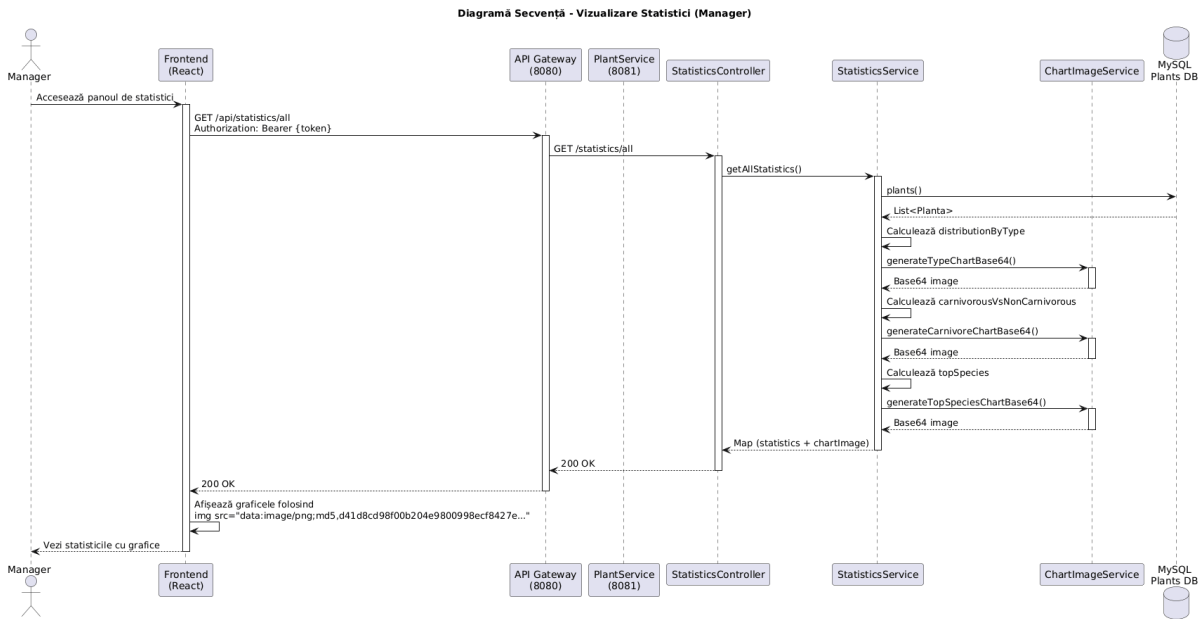
SALVARE STATISTICI



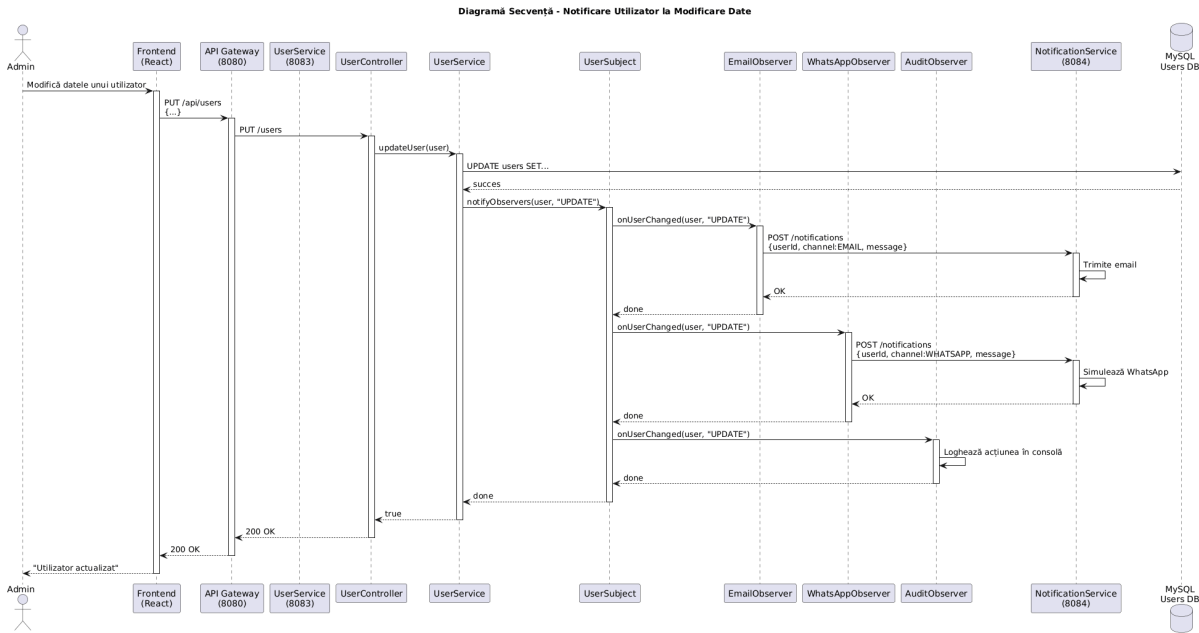
SALVARE UTILIZATORI



STATISTICI

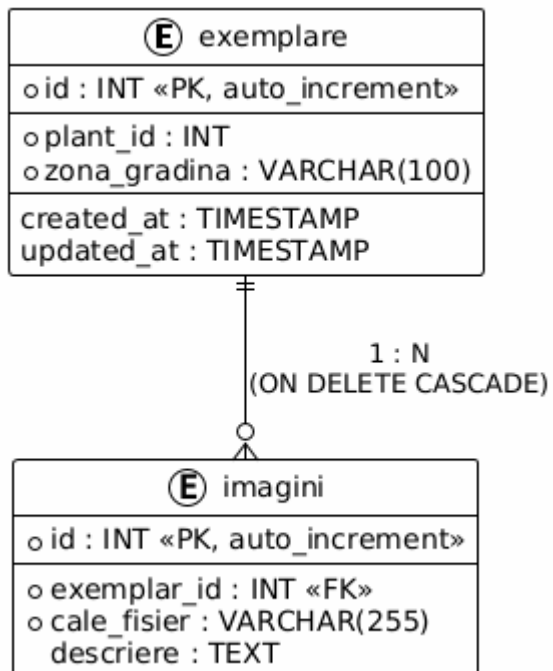
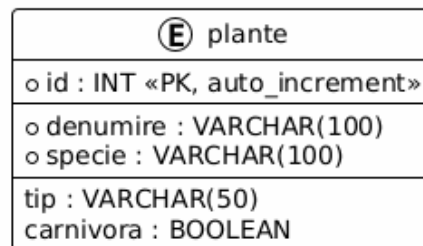


NOTIFICARE UTILIZATOR



DIAGREME ENTITY - RELATIONSHIP

Diagramă ER - Plant Service (bază de date b_plant)



Ⓔ users
o id : INT «PK, auto_increment»
o username : VARCHAR(100) «UK» o email : VARCHAR(150) «UK» o password_hash : VARCHAR(255) o role : ENUM(VISITOR,EMPLOYEE,MANAGER,ADMIN) o is_active : BOOLEAN o phone_number : VARCHAR(150)
created_at : TIMESTAMP

Ⓔ export_jobs
o id : INT «PK, auto_increment»
user_id : INT format : ENUM(CSV,JSON,XML,DOC) status : ENUM(PENDING,DONE,FAILED) file_path : VARCHAR(255) created_at : TIMESTAMP

Ⓔ notifications
o id : INT «PK, auto_increment»
user_id : INT channel : ENUM(EMAIL,WHATSAPP) message : TEXT status : ENUM(PENDING,SENT,FAILED) created_at : TIMESTAMP sent_at : TIMESTAMP error_message : TEXT

DIAGRAMA DE DEZVOLTARE

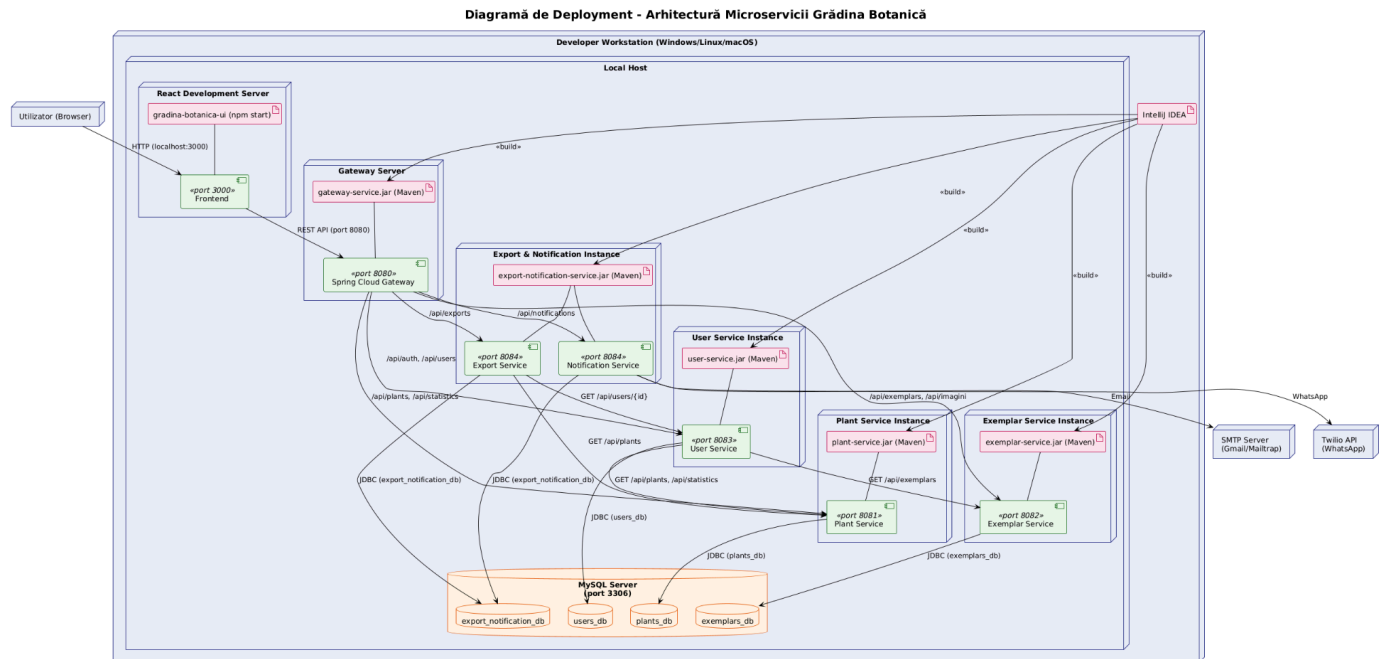
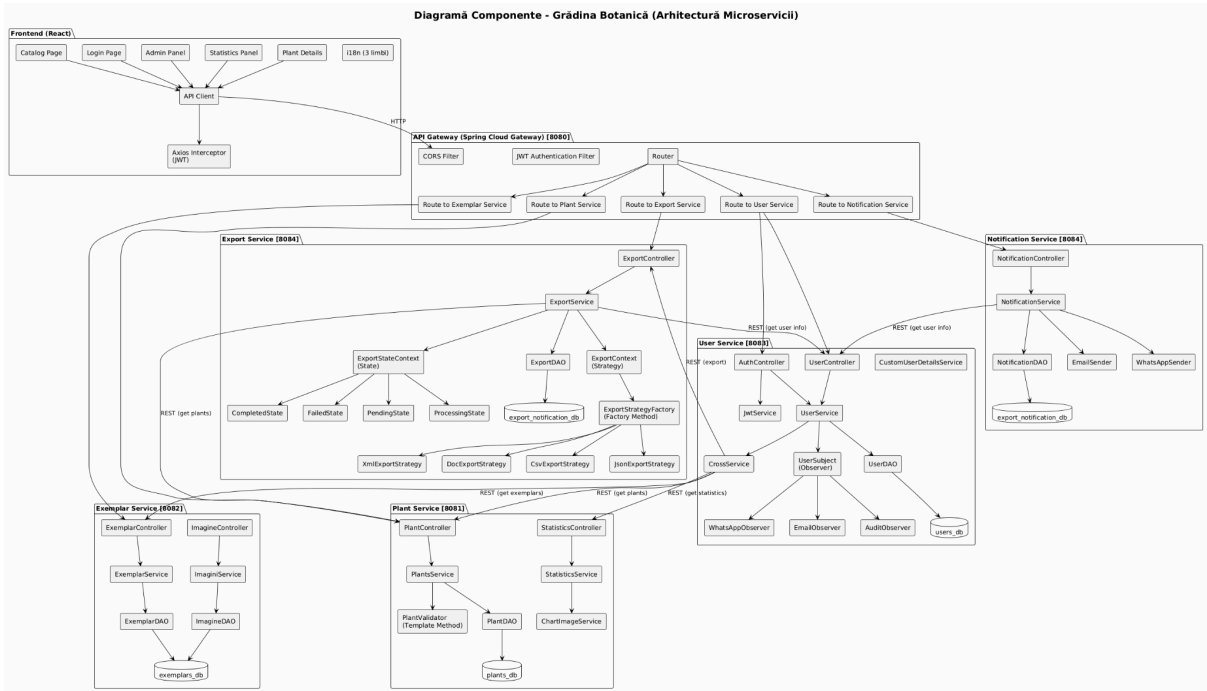


DIAGRAMA DE COMPONENTE



DIAGrame DE CLASE PENTRU CELE 4 MICROSERVICII

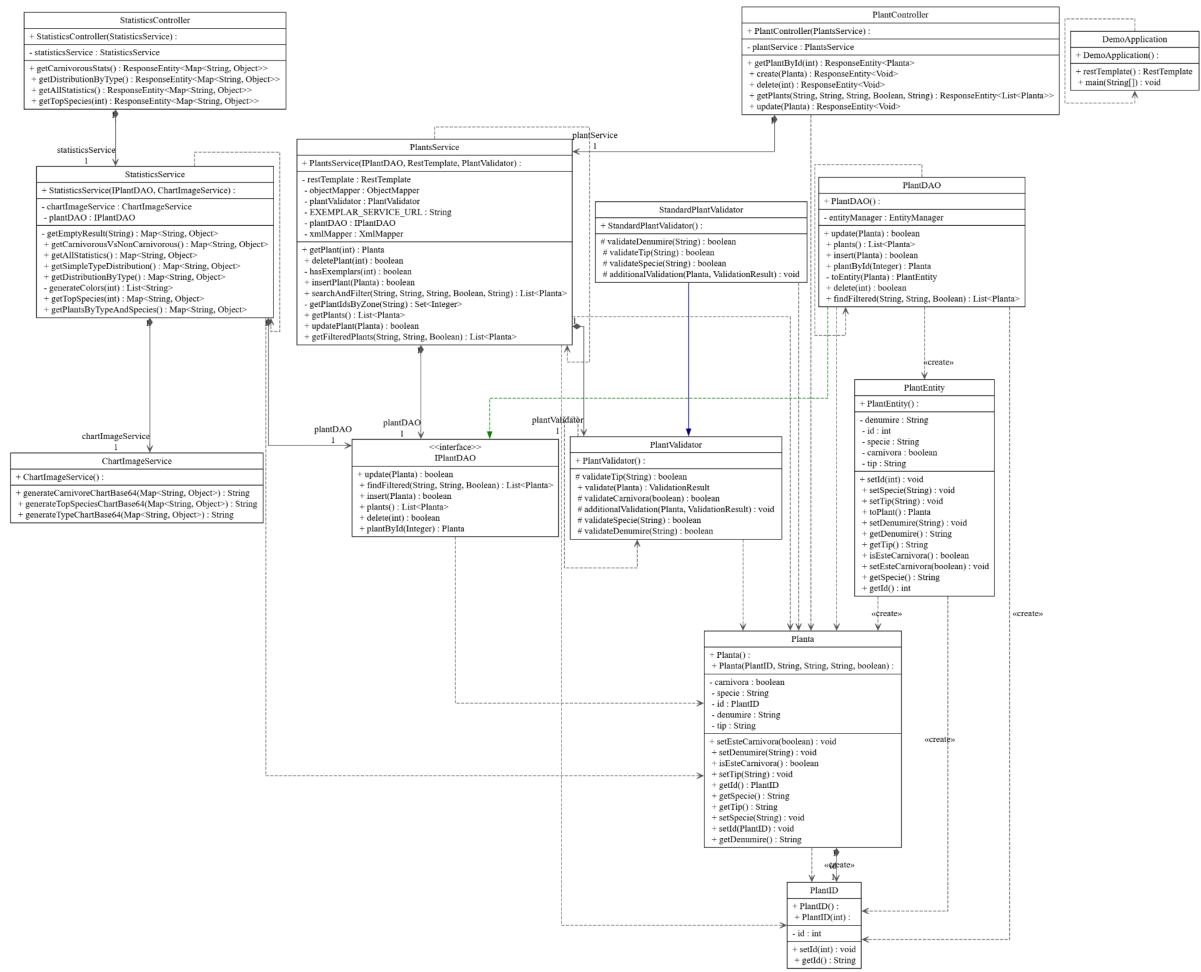
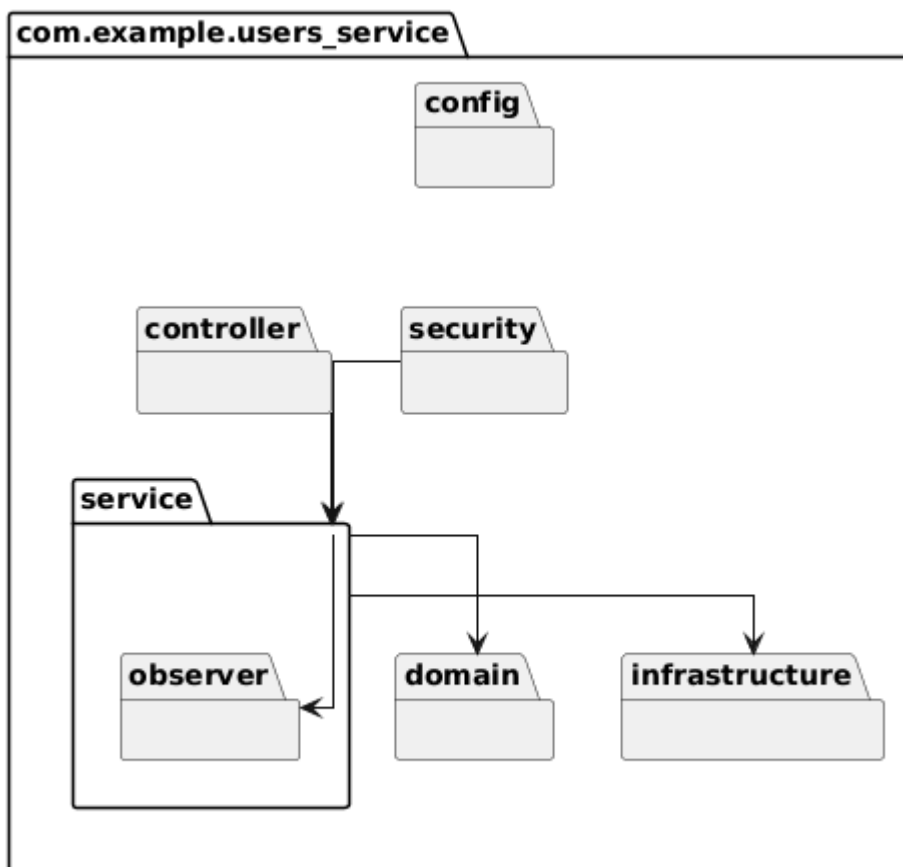
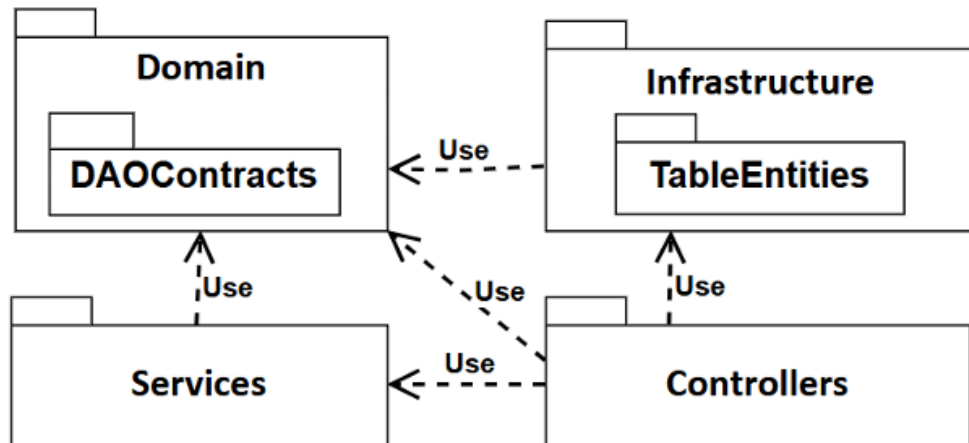


DIAGRAMME DE PACHETE



HARTA CONTEXT

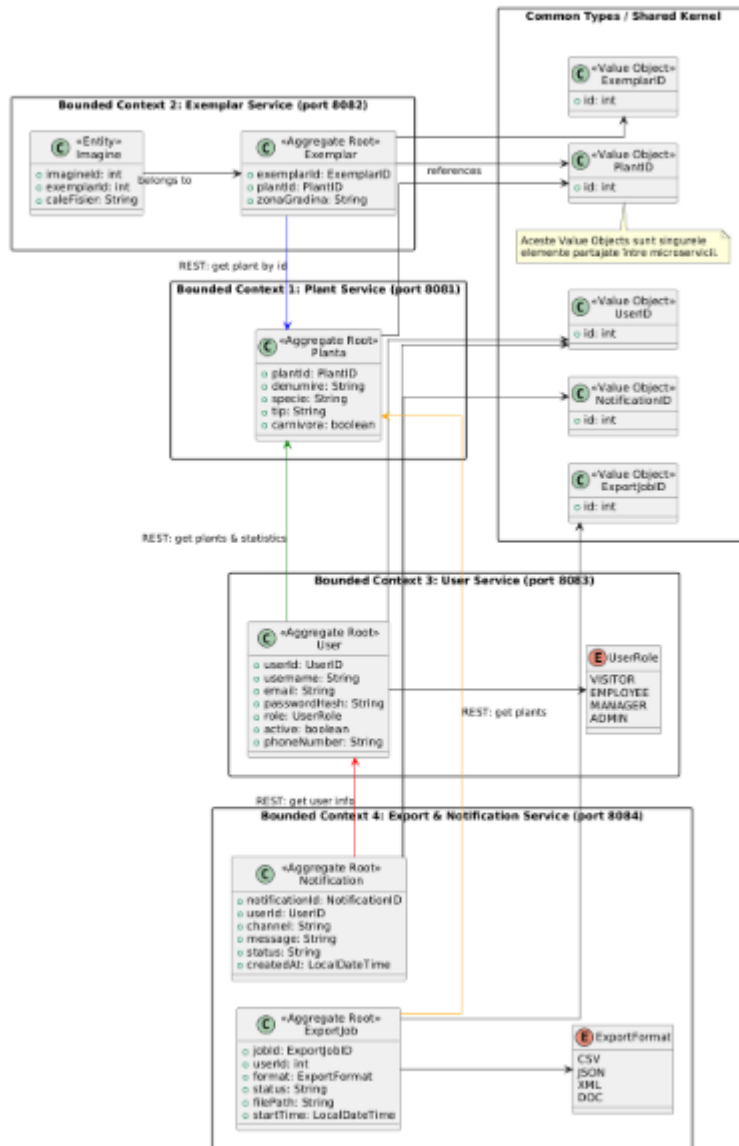


DIAGRAMA PACHETE FRONTEND

Diagramă de Pachete - Frontend (React)

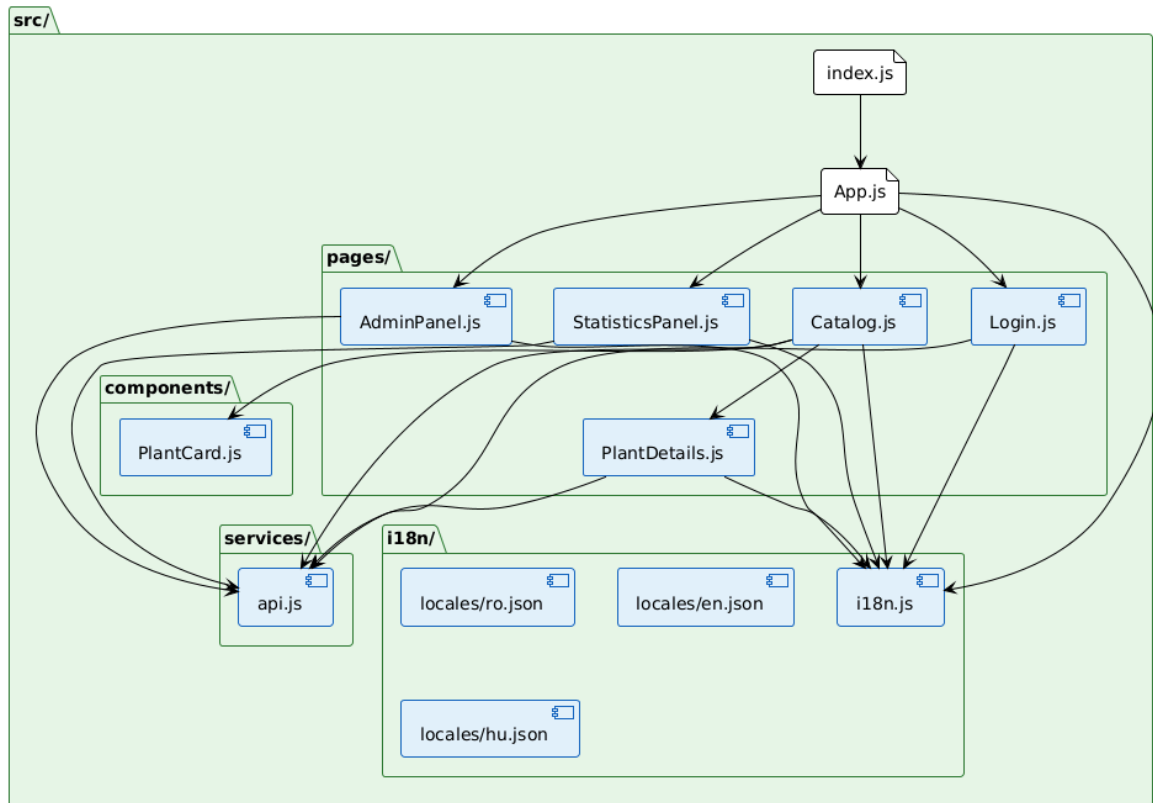


DIAGRAMA CLASE FRONTEND

